

СОДЕРЖАНИЕ

Введение.....	7
1 Обзор литературы	8
1.1 Анализ современных архитектурных подходов	8
1.2 Паттерны управления микросервисной инфраструктурой.....	9
1.3 Анализ технологий межсервисного взаимодействия	11
1.4 Сравнительный анализ систем управления базами данных и кэширования	13
1.5 Обзор решений для обеспечения безопасности и SSO	14
1.6 Анализ систем мониторинга и наблюдаемости	16
1.7 Выбор средств контейнеризации и развертывания	18
2 Системное проектирование.....	20
2.1 Блок API-шлюза	20
2.2 Блок аутентификации и авторизации.....	21
2.3 Блок обнаружения сервисов.....	21
2.4 Блок базовых микросервисов.....	22
2.5 Блок брокера сообщений	22
2.6 Блок мониторинга и наблюдаемости	23
2.7 Блок баз данных	23
2.8 Блок кэширования	24
3 Функциональное проектирование	25
3.1 Блок API-шлюза	25
3.2 Блок аутентификации и авторизации.....	27
3.3 Блок обнаружения сервисов.....	30
3.4 Блок базовых микросервисов.....	31
3.5 Блок брокера сообщений	42
3.6 Блок мониторинга и наблюдаемости	44
3.7 Блок баз данных	46
3.8 Блок кэширования	47
4 Разработка программных модулей	48
4.1 Реализация базовых микросервисов	48
4.2 Структура базового микросервиса	57
4.3 Описание REST API базовых микросервисов.....	59
5 Программа и методика испытаний.....	62
5.1 Конфигурация тестовой среды	62
5.2 Автоматизированное тестирование базовых микросервисов.....	63
5.3 Ручное тестирование REST API	64
5.4 Тестирование инфраструктурных компонентов	66
6 Руководство пользователя.....	70
7 Техничко-экономическое обоснование разработки и реализации на рынке инфраструктуры микросервисного приложения для агрегатора такси.....	71
7.1 Характеристика программного средства, разрабатываемого для реализации на рынке.....	71
7.2 Расчет инвестиций в разработку программного средства	72

7.3 Расчет экономического эффекта от реализации программного средства на рынке.....	75
7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке	77
7.5 Вывод об экономической целесообразности реализации проектного решения	77
Заключение	78
Список использованных источников	79
Приложение А	81
Приложение Б	82
Приложение В.....	83

ВВЕДЕНИЕ

В современном мире информационные технологии пронизывают практически все сферы человеческой деятельности, определяя скорость развития бизнеса, качество услуг и конкурентоспособность компаний. Одним из наиболее динамично развивающихся направлений в области разработки программного обеспечения является микросервисная архитектура – подход к построению приложений в виде набора небольших, независимых сервисов, каждый из которых отвечает за конкретную бизнес-функцию и может разрабатываться, развертываться и масштабироваться отдельно.

Микросервисная архитектура позволяет значительно повысить гибкость разработки, ускорить выход новых бизнес-ценностей на рынок, упростить поддержку и обновление систем, а также обеспечить высокую отказоустойчивость и масштабируемость при пиковых нагрузках. По данным различных исследований, в последние годы наблюдается устойчивый рост популярности этого подхода.

Особенно актуален микросервисный подход в высоконагруженных распределенных системах, таких как агрегаторы такси, маркетплейсы, финтех-сервисы и e-commerce платформы, где требуется обработка тысяч и миллионов запросов в реальном времени, высокая доступность и возможность независимого развития отдельных модулей.

Целью дипломного проектирования является разработка и развертывание инфраструктуры для микросервисного приложения агрегатора такси с базовой бизнес-логикой, обеспечивающей масштабируемость, отказоустойчивость и эффективное взаимодействие микросервисов.

Для достижения поставленной цели решаются следующие задачи:

- анализ современных практик и паттернов построения микросервисных систем;
- проектирование и настройка центральных инфраструктурных компонентов: шлюза, обнаружения сервисов, отказоустойчивости;
- организация системы единой авторизации и аутентификации;
- выбор и настройка хранилищ данных;
- реализация асинхронного взаимодействия между микросервисами;
- построение полноценной системы логирования и трассировки;
- разработка и настройка базовых микросервисов, реализующих стандартные операции.

Практическая значимость разработанной инфраструктуры заключается в создании масштабируемой, отказоустойчивой и удобной в эксплуатации инфраструктурной основы для высоконагруженных агрегаторов такси, что позволяет независимо развивать бизнес-функции (поиск водителя, расчет стоимости, оплата и т.д.), минимизировать простои и ускорять вывод новых возможностей на рынок.

Дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет **XX%** (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Анализ современных архитектурных подходов

На сегодняшний день выбор архитектурного стиля является определяющим фактором при проектировании масштабируемых информационных систем. Традиционным подходом долгое время являлась монолитная архитектура, при которой все компоненты приложения (бизнес-логика, работа с данными, пользовательский интерфейс) объединены в единый программный блок и развертываются как одно целое [1].

Основными достоинствами монолитной архитектуры являются простота разработки на начальных этапах, легкость тестирования и развертывания. Однако по мере роста системы, особенно в таких высоконагруженных доменах, как агрегаторы такси, монолит становится трудноуправляемым. Внесение изменений в один модуль требует сборки всего приложения, а невозможность масштабировать отдельные компоненты (например, только модуль расчета стоимости поездки) ведет к нерациональному использованию ресурсов.

В качестве альтернативы выступает микросервисная архитектура – подход, при котором приложение разбивается на набор слабосвязанных сервисов, взаимодействующих по сети. Каждый сервис отвечает за конкретную бизнес-способность и владеет собственной базой данных.

Разница между монолитной и микросервисной архитектурой представлена на рисунке 1.1.

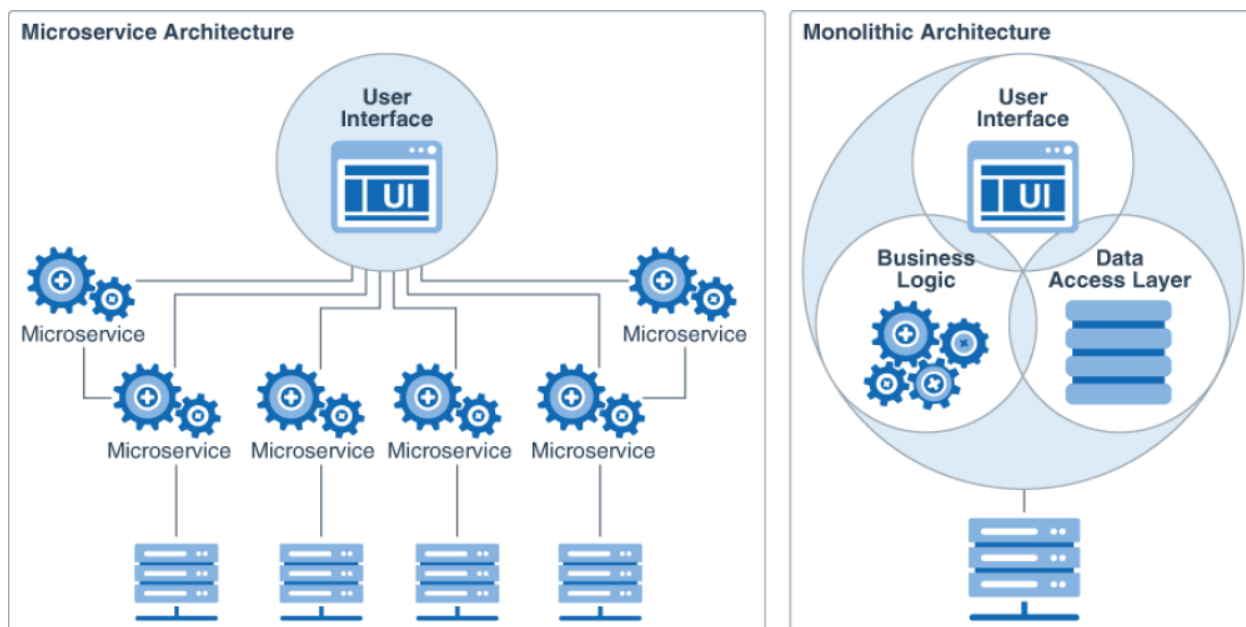


Рисунок 1.1 – Отличия монолитной и микросервисной архитектуры [1]

Согласно исследованию Криса Ричардсона [2], ключевыми преимуществами микросервисного подхода являются:

- независимость развертывания;
- возможность использования различных технологических стеков для разных задач;
- изоляция сбоев (отказ одного сервиса не приводит к полной остановке системы);
- легкое горизонтальное масштабирование наиболее нагруженных узлов.

Однако переход к микросервисам привносит сложность в управлении инфраструктурой. Возникают задачи обнаружения сервисов, балансировки нагрузки, обеспечения безопасности распределенных запросов и мониторинга состояния множества узлов.

1.2 Паттерны управления микросервисной инфраструктурой

Разделение приложения на десятки независимых сервисов порождает ряд проблем: как клиенту (мобильному приложению такси) узнать адреса всех сервисов, как обеспечить безопасность на входе и как предотвратить «эффект домино» при отказе одного из узлов. Для решения этих задач в современной практике проектирования выделяют три ключевых паттерна.

1.2.1 Паттерн API Gateway [3] представляет из себя наличие отдельного компонента, который выступает единой точкой входа для всех внешних запросов. Вместо того чтобы мобильное приложение или веб-интерфейс взаимодействовали с каждым микросервисом напрямую, они отправляют запрос на шлюз.

Помимо базовой маршрутизации, современный API Gateway выполняет ряд критически важных функций:

1 **Централизованная безопасность.** Шлюз берет на себя задачу терминирования SSL/TLS-соединений, избавляя внутренние микросервисы от необходимости обрабатывать сертификаты. Здесь же происходит проверка JWT-токенов или API-ключей. Это гарантирует, что неавторизованный трафик даже не попадет во внутренний контур сети.

2 **Ограничение частоты запросов.** Для агрегатора такси крайне важна защита от перегрузок или злонамеренных действий. Шлюз позволяет устанавливать лимиты на количество запросов в секунду от конкретного пользователя или IP-адреса.

3 **Кэширование ответов.** Для запросов, данные в которых меняются редко, шлюз может возвращать кэшированный ответ, значительно снижая нагрузку на нижележащие сервисы.

4 **Агрегация запросов.** Чтобы снизить количество сетевых вызовов от мобильного приложения, шлюз может принять один запрос, одновременно вызвать несколько микросервисов и вернуть клиенту объединенный результат.

Использование шлюза также упрощает мониторинг системы, так как именно здесь можно собирать общую статистику по времени ответа и

количеству ошибок во всей распределенной системе.

На рисунке 1.2 представлена схема взаимодействия сервисов через API Gateway.

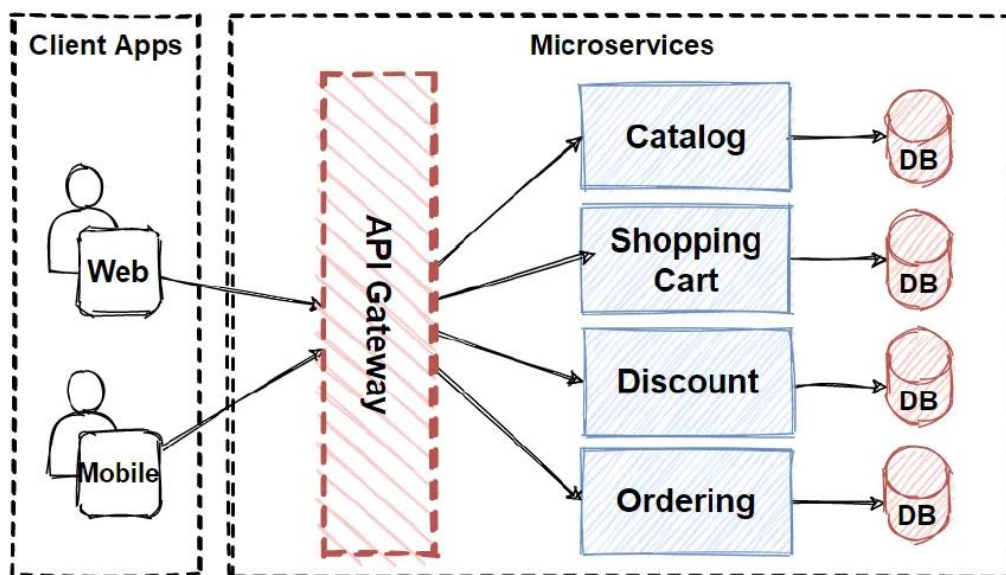


Рисунок 1.2 – Пример взаимодействия сервисов через API Gateway [3]

1.2.2 Паттерн Service Discovery [4] позволяет сервисам выполнять синхронные запросы без жестко прописанных IP-адресов в конфигурации микросервисов.

Система обнаружения состоит из двух частей: service registry (реестр), который представляет из себя базу данных, в которой хранится информация о доступных экземплярах сервисов и их адресах, и процесс регистрации, в котором микросервис сообщает реестру свой адрес, а при остановке – удаляет (рисунок 1.3).

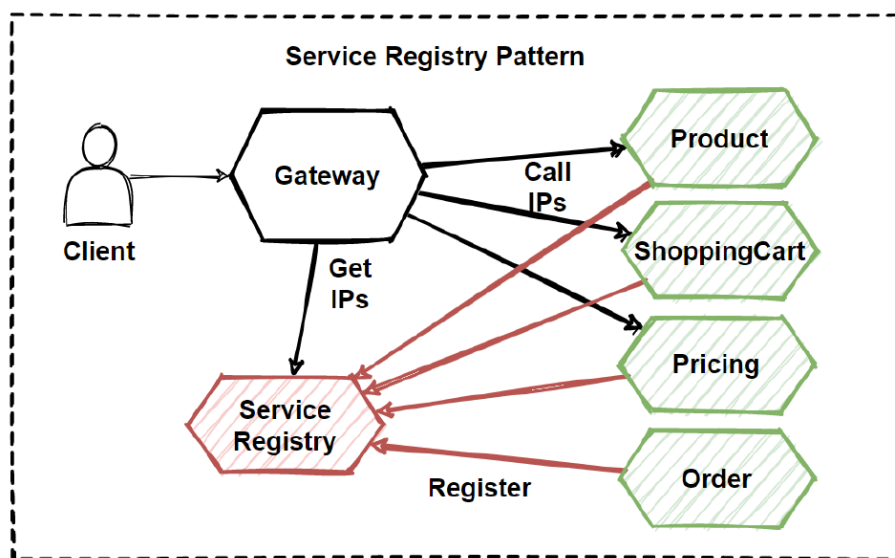


Рисунок 1.3 – Принцип работы паттерна Service Discovery [4]

Использование этого паттерна критически важно для приложений, где нагрузка может резко возрасти, что требует мгновенного запуска новых копий сервиса и их автоматического включения в работу системы.

1.2.3 Паттерн Circuit Breaker [5] позволяет легче справляться с отказами одного из сервисов в распределенной системе, так как отказы неизбежны. Если сервис А начинает отвечать с задержкой, запросы от сервиса В могут начать скапливаться, занимая память и потоки процессора, что в итоге приведет к падению всей системы. Данный паттерн позволяет избегать таких каскадных сбоев.

Паттерн работает по принципу электрического автомата и имеет три состояния (рисунок 1.4):

- closed (закрыт), когда запросы проходят в обычном режиме;
- open (открыт), если количество ошибок превышает порог, шлюз сразу возвращает ошибку клиенту, не пытаясь вызвать неисправный сервис;
- half-open (полуоткрыт), когда через некоторое время шлюз пропускает пробный запрос, чтобы проверить, восстановился ли сервис.

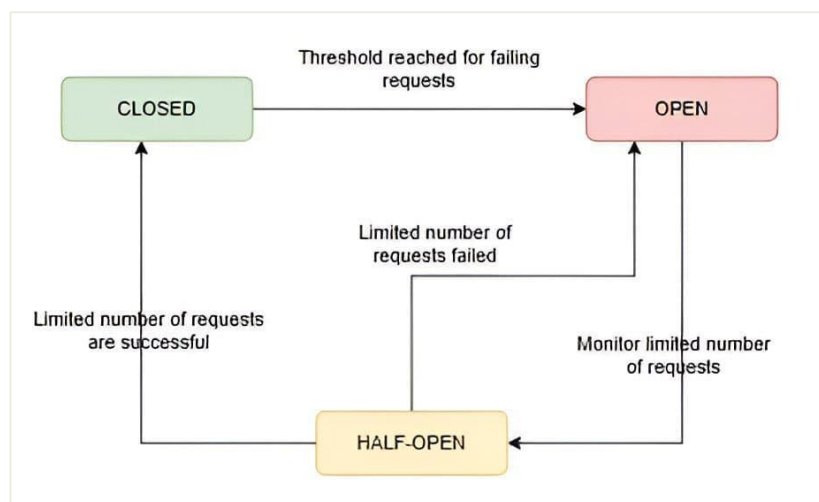


Рисунок 1.4 – Принцип работы паттерна Circuit Breaker [5]

Применение этого паттерна гарантирует, что даже если в приложении не работает хоть один сервис, пользователи все равно смогут продолжать работу с системой в целом, если неработающий сервис не затронул функционал, который необходим пользователю.

1.3 Анализ технологий межсервисного взаимодействия

В микросервисной архитектуре основное разделение взаимодействия между сервисами происходит на два типа: синхронное и асинхронное взаимодействие.

1.3.1 Синхронное взаимодействие подразумевает, что сервис-

отправитель ожидает немедленного ответа от сервиса-получателя. Наиболее распространенным протоколом здесь является REST (HTTP/JSON).

Преимущество синхронного подхода заключается в простоте реализации и отладки, и получение мгновенного результата выполнения операции.

Однако это создает большую связность между сервисами микросервисной системы, что негативно сказывается на общей производительности системы.

1.3.2 Асинхронное взаимодействие и событийно-ориентированная архитектура строится на передачи сообщений через промежуточное звено – брокер сообщений. В этом случае сервис-отправитель просто отправляет сообщение (событие) в очередь и продолжает свою работу, не дожидаясь обработки.

Сравнение двух способов взаимодействия представлено на рисунке 1.5.

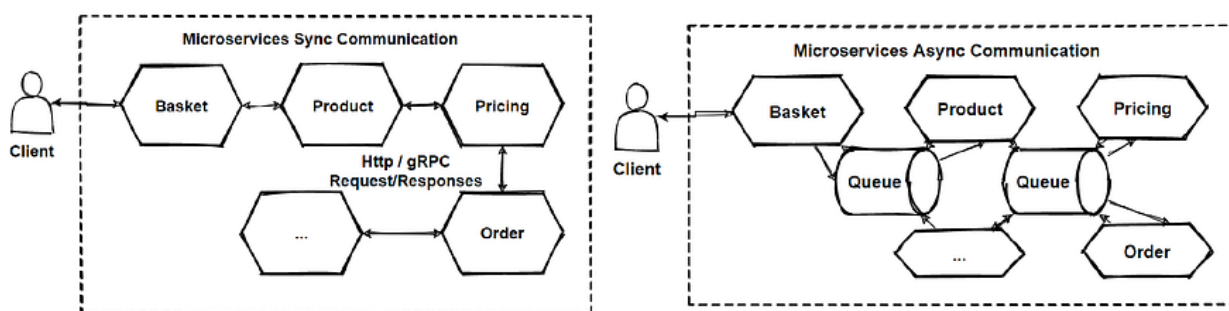


Рисунок 1.5 – Сравнение синхронного и асинхронного способа взаимодействия в микросервисной архитектуре [6]

Ключевые преимущества использования брокеров сообщений в крупных микросервисных инфраструктурах:

1 Сглаживание пиковых нагрузок. В праздничные дни количество заказов резко возрастает. Брокер выступает буфером, позволяя сервисам обрабатывать очередь сообщений с той скоростью, на которую они рассчитаны, не падая от перегрузки.

2 Гарантия доставки. Если сервис-обработчик временно отключен, сообщения накопятся в брокере и будут обработаны сразу после его включения.

3 Высокая пропускная способность в высоконагруженных системах.

4 Слабая связность. Сервис А ничего не знает о существовании сервиса В. Он публикует событие, а любые другие сервисы могут подписаться на это событие самостоятельно [6].

1.3.3 Технология Apache Kafka [7] используется для реализации асинхронного обмена данными в проекте. В отличие от классических очередей (например, RabbitMQ), Kafka является распределенной платформой потоковой передачи событий.

Основные отличия Kafka, важные для инфраструктуры агрегатора такси:

1 Высокая пропускная способность. Способность обрабатывать миллионы событий в секунду, что критично для отслеживания геолокации тысяч автомобилей в реальном времени.

2 Хранение данных. Kafka записывает сообщения на диск и хранит их в течение заданного времени. Это позволяет проводить «повтор событий» для восстановления состояния системы и проведения аналитики.

3 Масштабируемость. Система легко расширяется добавлением новых брокеров, что позволяет инфраструктуре расти вместе с базой пользователей агрегатора.

1.4 Сравнительный анализ систем управления базами данных и кэширования

В микросервисной архитектуре применяется паттерн «база данных на каждый сервис», что гарантирует слабую связность компонентов и независимость их масштабирования. Для агрегатора такси критически важными требованиями к хранилищам данных являются обеспечение строгой консистентности финансовых транзакций (оплата поездок), надежное хранение профилей пользователей, а также возможность быстрой обработки геопространственных данных.

1.4.1 В качестве основной СУБД выступает PostgreSQL [8], которая является объектно-реляционной системой. Ключевыми преимуществами PostgreSQL для инфраструктуры агрегатора такси являются:

1 Строгое соответствие принципам ACID (атомарность, согласованность, изолированность, долговечность), что критически важно для микросервисов, отвечающих за биллинг и расчеты.

2 Расширенная поддержка JSON/JSONB. Это позволяет гибко хранить слабоструктурированные данные (например, логи изменений статусов заказов) в рамках реляционной модели, частично заменяя необходимость использования NoSQL-решений (таких как MongoDB).

3 Наличие расширения PostGIS. Данное расширение делает PostgreSQL мощным инструментом для обработки геопространственных данных, (что является ядром бизнес-логики любого агрегатора такси).

Помимо надежного хранения, в высоконагруженных системах возникает необходимость минимизации времени отклика и снижения нагрузки на основную БД при частых запросах одних и тех же данных (например, тарифы, текущие сессии пользователей, статусы активности водителей). Для этого применяются системы кэширования в оперативной памяти (In-Memory Data Grid).

Наиболее распространенными решениями в этой области являются Memcached и Redis.

Memcached представляет собой простую и высокопроизводительную распределенную систему кэширования объектов в памяти на основе хэш-

таблиц (ключ-значение). Она отлично справляется с базовым кэшированием строковых данных, но не поддерживает сложные структуры данных.

Redis, в отличие от Memcached, является сервером структур данных. Его основные преимущества для проектируемой системы:

1 Поддержка сложных типов данных: списков, множеств, хэшей, что позволяет реализовывать сложные алгоритмы кэширования на стороне БД.

2 Встроенная поддержка геопространственных индексов, что позволяет хранить и мгновенно обновлять текущие координаты водителей в оперативной памяти, не перегружая дисковую подсистему PostgreSQL.

3 Возможность сохранения данных на диск, что обеспечивает быстрое восстановление состояния кэша после перезагрузки узла.

На основе проведенного сравнительного анализа для разрабатываемой микросервисной инфраструктуры в качестве основной реляционной СУБД для хранения бизнес-сущностей выбрана PostgreSQL, а в качестве in-memory хранилища для кэширования и управления быстроменяющимися состояниями – Redis. Данная связка является индустриальным стандартом и обеспечивает необходимый баланс между надежностью, масштабируемостью и производительностью.

1.5 Обзор решений для обеспечения безопасности и SSO

В микросервисной архитектуре агрегатора такси проблема аутентификации и авторизации требует особого подхода. Поскольку система состоит из множества независимых сервисов, передача учетных данных пользователя (логина и пароля) каждому из них небезопасна и неэффективна. Для решения этой задачи применяется концепция единого входа (Single Sign-On, SSO) и централизованного управления доступом. Основными стандартами для реализации безопасного делегированного доступа на сегодняшний день являются протоколы OAuth 2.0 и OpenID Connect (OIDC) [9].

1.5.1 Облачные провайдеры аутентификации (IDaaS) [10] – сервисы, предоставляющие функционал управления доступом по модели SaaS. Наиболее популярными представителями являются Auth0, Firebase Authentication и AWS Cognito. Данные платформы берут на себя всю сложность реализации криптографии, хранения паролей и масштабирования.

Сервисы данного типа обладают следующими достоинствами:

- максимально быстрое внедрение и интеграция;
- готовый пользовательский интерфейс для регистрации, входа и восстановления пароля;
- высокая отказоустойчивость, обеспечиваемая поставщиком услуг;
- встроенные механизмы защиты от атак и многофакторная аутентификация.

Однако облачные сервисы имеют существенные недостатки:

- хранение персональных данных пользователей на сторонних, зачастую зарубежных серверах, что может нарушать законодательные требования о

локализации данных;

- привязка к экосистеме конкретного провайдера;
- высокая стоимость эксплуатации при росте активной пользовательской базы (оплата за количество уникальных пользователей).

1.5.2 Фреймворки для создания собственных серверов авторизации [11]. Если проект требует уникальной логики аутентификации, разработчики могут реализовать сервер с нуля, используя специализированные библиотеки, такие как IdentityServer (для экосистемы .NET) или Spring Security Authorization Server (для Java).

Достоинства подхода:

- полный контроль над логикой работы, алгоритмами шифрования и процессом выдачи токенов;
- возможность интеграции с любыми устаревшими базами данных.

Недостатки данного подхода:

- колоссальные временные затраты на разработку, тестирование и поддержку;
- высокие риски допущения критических уязвимостей в безопасности из-за человеческого фактора;
- необходимость самостоятельной разработки панелей администрирования пользователей.

1.5.3 Готовые решения для локального развертывания. Этот подход объединяет преимущества облачных платформ (готовый функционал) и собственной разработки (полный контроль над данными). Самым популярным решением в этой категории является Keycloak [12] – продукт с открытым исходным кодом, разрабатываемый при поддержке Red Hat.

Keycloak представляет собой полноценный сервер аутентификации и авторизации, который разворачивается в собственной инфраструктуре проекта.

Его основные достоинства:

- бесплатное использование без ограничений на количество пользователей;
- поддержка всех современных протоколов (OAuth 2.0, OpenID Connect, SAML);
- готовая реализация SSO, позволяющая пользователям авторизоваться один раз для доступа ко всем подсистемам агрегатора;
- гибкая настройка ролевой модели доступа (RBAC) и панель администратора «из коробки»;
- полный контроль над данными, так как база данных с учетными записями находится во внутреннем изолированном контуре инфраструктуры и недоступна извне.

В таблице 1.3 представлен сравнительный анализ рассмотренных подходов, который позволяет выбрать оптимальный подход к вопросам аутентификации и авторизации.

Таблица 1.1 – Сравнение решений для обеспечения безопасности и SSO

Решение	Тип	Достоинства	Недостатки
Auth0 или Firebase	Облачный сервис (SaaS)	Быстрый старт, высокая надежность, не требует поддержки серверов	Риски хранения персональных данных у третьих лиц, высокая цена при масштабировании
IdentityServer или Spring Authorization Server	Библиотека	Абсолютный контроль над логикой, глубокая кастомизация	Высокая стоимость разработки, риск уязвимостей, требует узких специалистов
Keycloak	Готовый сервер	Бесплатный функционал SSO «из коробки», полный контроль над данными	Требует ресурсов для локального хостинга и настройки инфраструктуры

Таким образом, в контексте разработки микросервисной инфраструктуры для агрегатора такси, где критически важны безопасность, независимость от сторонних вендоров и защита персональных данных, подход с использованием сторонних облачных API нецелесообразен. Разработка собственного сервера требует неоправданно высоких трудозатрат. Решение Keycloak является наиболее сбалансированным выбором, предоставляя стандартизированный функционал уровня Enterprise при полном контроле над развертыванием и хранением данных.

1.6 Анализ систем мониторинга и наблюдаемости

Разделение монолитного приложения на десятки независимых микросервисов неизбежно усложняет процессы поиска неисправностей и контроля состояния системы. В случае сбоя администраторам сложно определить, в каком именно узле или базе данных произошла ошибка. Для решения этой проблемы внедряется концепция «наблюдаемости» (Observability), базирующаяся на трех основных компонентах: метриках, логировании и распределенной трассировке.

1.6.1 Системы сбора метрик представляют из себя инструменты, которые собирают числовые показатели работы системы (потребление CPU, оперативной памяти, количество HTTP-запросов в секунду).

Традиционным решением для мониторинга серверов является система Zabbix. Она использует push-модель, при которой агенты, установленные на серверах, регулярно отправляют данные на центральный сервер. Zabbix

отлично подходит для статической инфраструктуры, однако в динамических микросервисных средах (где Docker-контейнеры могут создаваться и удаляться ежеминутно) управление агентами становится затруднительным.

В качестве индустриального стандарта для контейнеризированных приложений выступает Prometheus [13]. Его ключевым отличием является pull-модель: сервер мониторинга самостоятельно опрашивает метрики с микросервисов по HTTP-протоколу. Prometheus обладает встроенной базой данных временных рядов и мощным языком запросов PromQL, что делает его оптимальным выбором для агрегатора такси.

1.6.2 Системы централизованного логирования представляют из себя системы, которые содержат детальную текстовую информацию о событиях внутри приложения. Поскольку микросервисы распределены по разным контейнерам, необходимо собирать их логи в единое хранилище.

Исторически самым популярным решением является стек ELK (Elasticsearch, Logstash, Kibana) [14]. Elasticsearch обеспечивает мощный полнотекстовый поиск по всем логам, а Kibana предоставляет удобный веб-интерфейс для аналитики (рисунок 1.6).

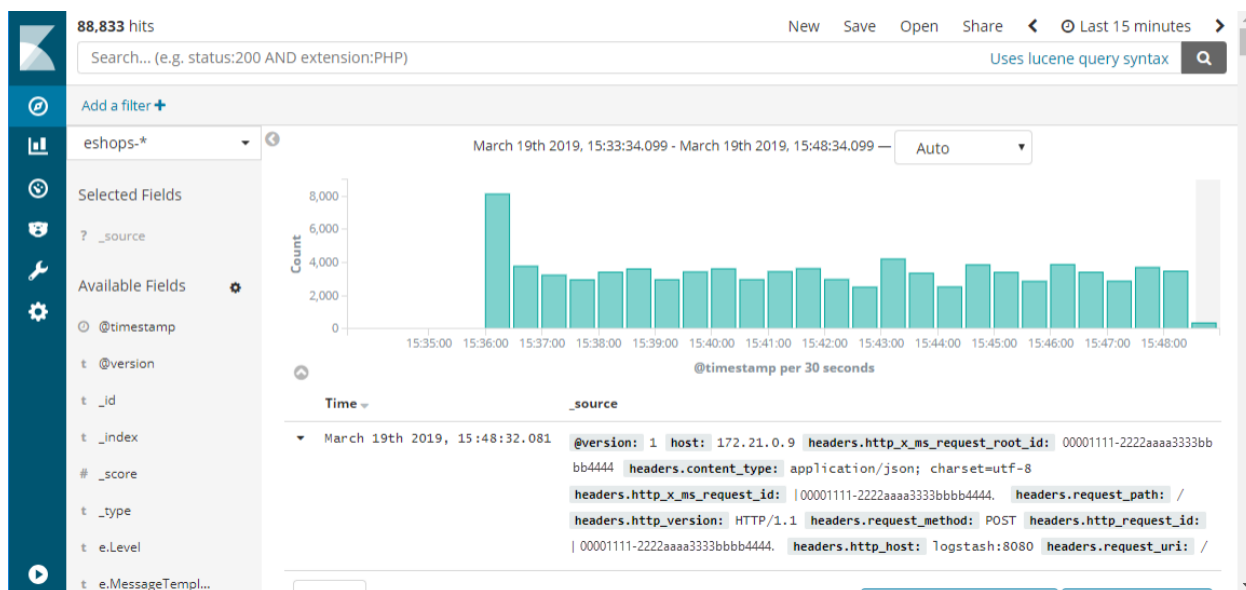


Рисунок 1.6 – Веб-интерфейс Kibana

Несмотря на высокую функциональность, главным недостатком стека ELK является колоссальное потребление вычислительных ресурсов. Индексация полного текста каждого сообщения требует значительных объемов оперативной памяти, что избыточно для базовой инфраструктуры.

Современной и легковесной альтернативой является стек LOKI [15], разработанный компанией Grafana Labs. В отличие от Elasticsearch, Loki не индексирует содержимое логов, а индексирует только метаданные, аналогично подходу Prometheus. Текст логов сжимается и хранится в объектном хранилище. Это позволяет кардинально снизить требования к

ресурсам серверов (CPU и RAM), сохраняя при этом высокую скорость поиска по нужным ярлыкам. Веб-интерфейс Grafana представлен на рисунке 1.7.

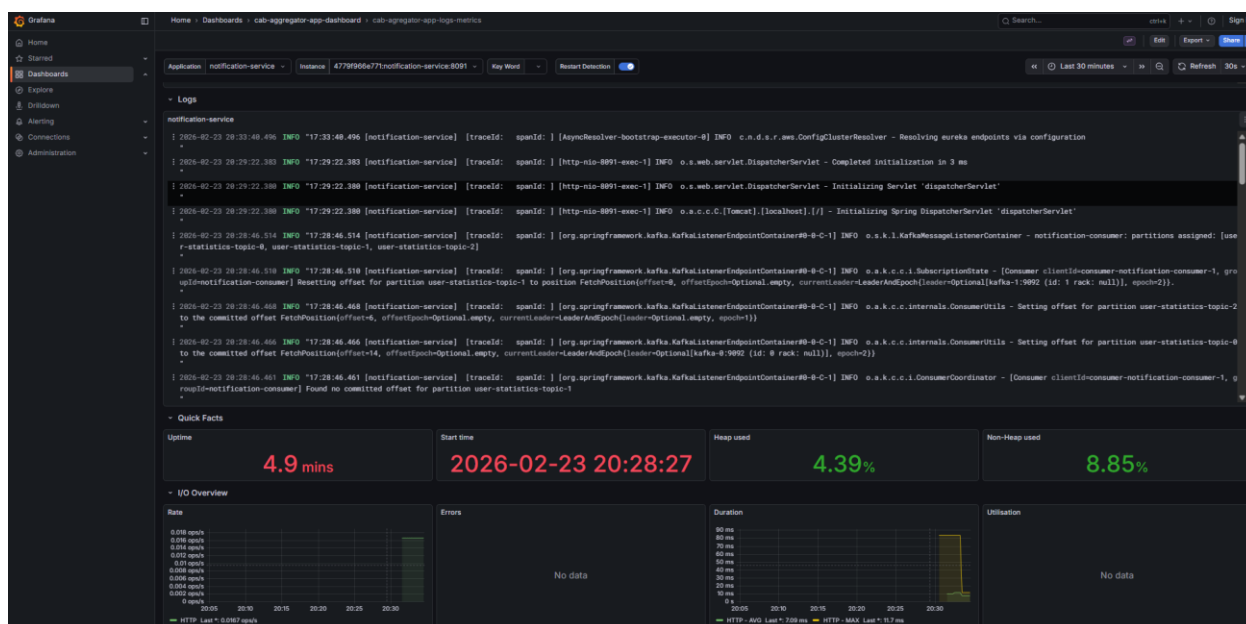


Рисунок 1.7 – Веб-интерфейс Grafana

1.6.3 Система распределенной трассировки позволяет отследить путь одного пользовательского запроса и выявить узкие места производительности. Ранее для этих целей использовались системы Jaeger или Zipkin со своими проприетарными агентами. На сегодняшний день стандартом де-факто стал проект OpenTelemetry [16] – единый набор API и SDK для инструментирования кода. OpenTelemetry позволяет собирать трейсы и отправлять их в любой backend.

В качестве хранилища трейсов для проекта выбрана система Tempo. Она глубоко интегрирована с Loki и Prometheus, отличается высокой скоростью записи и не требует сложной настройки баз данных для своей работы.

Исходя из всей информации выше оптимальным решением в данной инфраструктуре является использование LOKI стека, как более современный и легковесный, при этом не уступающий по функционалу ELK стеку.

1.7 Выбор средств контейнеризации и развертывания

Разработка масштабируемой микросервисной архитектуры агрегатора такси, включающей базы данных (PostgreSQL, Redis), брокер сообщений (Kafka), сервер авторизации (Keycloak) и системы мониторинга, требует унифицированного подхода к развертыванию. Главная проблема при переносе такого количества компонентов между средами заключается в конфликтах зависимостей и различиях в конфигурации операционных систем.

Для решения этой проблемы применяются технологии изоляции сред выполнения. На сегодняшний день выделяют несколько основных подходов к

развертыванию информационных систем.

1.7.1 В отличие от виртуальных машин, контейнеризация на уровне ОС не эмулирует аппаратное обеспечение, а используют общее ядро операционной системы хоста, изолируя процессы на уровне пространств имен (namespaces) и контрольных групп (cgroups) Linux. Это позволяет запускать сервисы за миллисекунды и потреблять минимум ресурсов.

Среди платформ контейнеризации рассматриваются две основные:

1 Podman [17] – современная утилита от Red Hat. Ее главное преимущество заключается в архитектуре без демона и возможности запуска контейнеров без root-прав, что повышает безопасность. Однако инструмент имеет более сложную настройку сетевого взаимодействия и меньшее количество готовых интеграций.

2 Docker [18] – индустриальный стандарт де-факто для контейнеризации. Docker предоставляет удобный CLI-интерфейс, огромную экосистему готовых образов (Docker Hub) и отличается высокой стабильностью. В контексте данной инфраструктуры, где требуется объединить множество готовых инфраструктурных решений, Docker является наиболее надежным и задокументированным инструментом.

1.7.2 Средства оркестрации контейнеров позволяют управлять многоконтейнерными приложениями на разных узлах, при этом описывая этот процесс декларативно.

Для решения этой задачи рассматриваются системы оркестрации:

1 Kubernetes [19] – мощнейшая платформа для управления кластерами контейнеров. Она обеспечивает автоматическое масштабирование, самовосстановление и балансировку нагрузки. Недостатком K8s является чрезвычайно высокий порог входа, сложность архитектуры и высокие требования к ресурсам. Использование Kubernetes для базового развертывания инфраструктуры на текущем этапе является избыточным решением.

2 Docker Compose [20] – утилита для локального развертывания и связывания контейнеров с использованием формата YAML. С помощью одного конфигурационного файла docker-compose.yaml и одной команды можно развернуть всю спроектированную инфраструктуру агрегатора со всеми зависимостями.

Важнейшим архитектурным преимуществом выбора формата Docker Compose является его прямая совместимость с нативным оркестратором Docker Swarm [21]. Если в будущем проекту потребуется распределение нагрузки между несколькими физическими серверами (создание кластера), конфигурационные файлы docker-compose.yaml имеют полную обратную совместимость с Docker Swarm.

Для упаковки всех компонентов приложения выбран Docker, а в качестве инструмента декларативного описания и запуска инфраструктуры – Docker Compose с перспективой прозрачного масштабирования через Docker Swarm оркестратор.

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

Проектирование инфраструктуры для агрегатора такси основывается на микросервисной архитектуре, что обеспечивает высокую масштабируемость, отказоустойчивость и гибкость в разработке и развертывании. Кроме того, микросервисная архитектура значительно повышает отказоустойчивость, так как сбой одного компонента не приводит к полному отказу всей системы. Гибкость в разработке и развертывании также является ключевым преимуществом, позволяя командам работать независимо друг от друга.

Было выделено несколько блоков, каждый из которых отвечает за определенные функции в этой системе:

- блок API-шлюза;
- блок аутентификации и авторизации;
- блок обнаружения сервисов;
- блок базовых микросервисов;
- блок брокера сообщений;
- блок мониторинга и наблюдаемости;
- блок баз данных;
- блок кэширования.

Это структурное разделение позволяет управлять всеми аспектами работы агрегатора такси, обеспечивая независимое развитие, тестирование и масштабирование каждого компонента. Взаимосвязи между блоками представлены на структурной схеме ГУИР.400201.047 С1.

2.1 Блок API-шлюза

Блок API-шлюза является единой точкой входа для всех внешних запросов к системе агрегатора такси. Он выступает в роли фасадного слоя, который принимает запросы от клиентских приложений и маршрутизирует их к соответствующим внутренним микросервисам. Это позволяет скрыть сложность внутренней архитектуры от внешних клиентов и обеспечить централизованное выполнение ряда ключевых функций.

Основные функции блока API-шлюза:

- маршрутизация запросов;
- централизованное обеспечение безопасности;
- кэширование ответов;
- агрегация запросов.

Размещение API-шлюза как изолированного компонента (микросервиса) позволяет ему масштабироваться независимо от других сервисов. Использование такого паттерна, как API Gateway [3], является стандартом для микросервисных архитектур, обеспечивая гибкость, безопасность и управляемость высоконагруженных систем. Таким образом блок API-шлюза является неотъемлемой частью инфраструктуры для приложения агрегатора такси, предоставляя единую точку входа и централизованное управление запросами через шлюз.

2.2 Блок аутентификации и авторизации

Блок аутентификации и авторизации отвечает за проверку подлинности пользователей и предоставление им соответствующих прав доступа к ресурсам агрегатора такси. В условиях микросервисной архитектуры крайне важно централизовать этот процесс, чтобы избежать дублирования логики и обеспечить единую политику безопасности.

Основные задачи данного блока:

- аутентификация пользователей;
- управление пользователями;
- авторизация доступа;
- управление сессиями;
- защита данных.

Для реализации данного блока будет использовано готовое решение для локального развертывания – Keycloak [12]. Keycloak является продуктом с открытым исходным кодом, который поддерживает все современные протоколы (OAuth 2.0, OpenID Connect, SAML) и предоставляет:

- единый вход (SSO);
- гибкую ролевую модель;
- панель администратора;
- полный контроль над данными.

Использование Keycloak позволяет избежать колоссальных временных затрат на разработку собственной системы аутентификации и авторизации, при этом обеспечивая высокий уровень безопасности и гибкость в управлении доступом.

2.3 Блок обнаружения сервисов

Блок обнаружения сервисов (Service Discovery) является фундаментальным элементом микросервисной архитектуры, позволяющим сервисам динамически находить друг друга без жесткой привязки к статическим IP-адресам или портам в конфигурации. В распределенной системе, где микросервисы постоянно запускаются, останавливаются и масштабируются, Service Discovery обеспечивает актуальность информации о доступных экземплярах.

Основные функции блока обнаружения сервисов:

- динамическая регистрация сервисов в реестре;
- поиск сервисов;
- балансировка нагрузки;
- устойчивость к отказам.

Применение обнаружение сервисов критически важно для систем с высокой динамичностью, таких как агрегатор такси, где нагрузка может резко меняться, требуя быстрого масштабирования сервисов. Этот подход упрощает управление инфраструктурой, повышает надежность и избавляет разработчиков от необходимости вручную управлять сетевыми адресами.

2.4 Блок базовых микросервисов

Блок базовых микросервисов представляет собой набор независимых, слабосвязанных сервисов, каждый из которых инкапсулирует конкретную бизнес-логику агрегатора такси. Это основной функциональный уровень системы, отвечающий за выполнение ключевых операций.

Для того, чтобы построить инфраструктуру для микросервисной архитектуры, необходимо иметь базовые операции (CRUD) в базовых доменных сервисах.

На стартовом этапе в приложении агрегаторе такси можно выделить следующие сервисы в блоке базовых микросервисов:

- сервис водителей;
- сервис пассажиров;
- сервис рейтинга;
- сервис поездок;
- сервис регистрации;
- сервис уведомлений.

Для последующего масштабирования сервисов достаточно в данный блок добавить новый сервис с бизнес-логикой, который будет автоматически включаться и интегрироваться с другими компонентами системы.

Разделение системы на базовые микросервисы значительно повышает гибкость разработки, ускоряет вывод новых функций на рынок, упрощает поддержку и обеспечивает высокую отказоустойчивость. В случае сбоя одного сервиса, остальные продолжают функционировать, минимизируя влияние на всю систему.

2.5 Блок брокера сообщений

Блок брокера сообщений играет центральную роль в реализации асинхронного взаимодействия между микросервисами в архитектуре агрегатора такси. Он позволяет сервисам обмениваться данными и событиями без прямой синхронной связи, что значительно повышает отказоустойчивость, масштабируемость и слабую связанность системы.

Можно выделить следующие основные функции блока брокера сообщений:

- асинхронный обмен данными;
- сглаживание пиковых нагрузок;
- гарантия доставки;
- слабая связанность;
- поддержка событийной архитектуры.

Для реализации брокера сообщений в проекте будет использоваться Apache Kafka [7]. Данный брокер сообщений обладает высокой пропускной способностью и возможностью хранения сообщений. Горизонтальное масштабирование позволяет легко добавлять новые экземпляры брокеров сообщений без дополнительной конфигурации.

2.6 Блок мониторинга и наблюдаемости

Блок мониторинга и наблюдаемости предназначен для сбора, анализа и визуализации данных о состоянии и производительности всех компонентов распределенной системы агрегатора такси. В условиях микросервисной архитектуры важно иметь полное представление об их работе для оперативного выявления и устранения проблем.

Данный блок базируется на концепции «Observability» (наблюдаемость), которая включает три основных компонента: метрики, логирование и распределенная трассировка.

Основные задачи блока мониторинга и наблюдаемости:

- сбор метрик, логов и трассировок со всех микросервисов;
- хранение оперативных данных;
- анализ и корреляция оперативных данных;
- визуализация работоспособности отдельных микросервисов.

Для реализации мониторинга и наблюдаемости будет использоваться LOKI стек в связке с Prometheus и Tempo:

1 Prometheus [13]: сбор метрик. Использует pull-модель, самостоятельно опрашивая метрики с микросервисов по HTTP-протоколу. Обладает встроенной базой данных временных рядов и мощным языком запросов PromQL.

2 Loki [15]: централизованное логирование. В отличие от традиционных решений, Loki индексирует только метаданные логов, а не их содержимое, что значительно снижает требования к вычислительным ресурсам при сохранении высокой скорости поиска.

3 Tempo [16]: распределенная трассировка. Глубоко интегрирован с Loki и Prometheus, обеспечивает высокую скорость записи и не требует сложной настройки.

4 Grafana [14]: веб-интерфейс для визуализации всех данных (метрик, логов, трассировок) из Prometheus, Loki и Tempo.

Использование LOKI стека является современным и легковесным решением, которое обеспечивает высокий уровень наблюдаемости для инфраструктуры приложения агрегатора такси.

2.7 Блок баз данных

Блок баз данных является фундаментальным компонентом архитектуры агрегатора такси, отвечающим за надежное хранение всей персистентной информации. В микросервисной архитектуре применяется паттерн «база данных на каждый сервис», что обеспечивает слабую связанность компонентов и независимость их масштабирования.

Для агрегатора такси критически важны требования к хранилищам данных, такие как:

- строгая согласованность данных;
- надежное хранение данных;

- быстрая обработка геопространственных данных;
- гибкость данных.

В качестве основной СУБД будет использоваться PostgreSQL [8], так как она строго соответствует принципам ACID, имеет расширенную поддержку JSON/JSONB и имеет наличие расширения PostGIS для работы с геопространственными данными.

Каждый базовый микросервис будет использовать свою собственную базу данных PostgreSQL, что позволит ему развиваться и масштабироваться независимо. Такой подход обеспечивает изоляцию данных, предотвращает взаимозависимости и упрощает управление схемами данных для каждого сервиса.

2.8 Блок кэширования

Блок кэширования предназначен для минимизации времени отклика и снижения нагрузки на основные базы данных за счет временного хранения часто запрашиваемых данных в оперативной памяти. В высоконагруженных системах, таких как агрегатор такси, кэширование является критически важным для обеспечения высокой производительности и масштабируемости.

Типовые данные, которые могут быть кэшированы:

- тарифы;
- текущие сессии пользователей;
- статусы активности водителей.

Для реализации in-memory хранилища для кэширования и управления быстро изменяющимися состояниями будет использоваться Redis. Преимущества Redis по сравнению с другими решениями (например, Memcached):

1 Поддержка сложных типов данных. Redis поддерживает списки, множества, хэши, что позволяет реализовывать более сложные алгоритмы кэширования и решать специфические задачи.

2 Встроенная поддержка геопространственных индексов. Позволяет эффективно хранить и мгновенно обновлять текущие координаты водителей в оперативной памяти, что снимает нагрузку с PostgreSQL.

3 Возможность сохранения данных на диск. Обеспечивает быстрое восстановление состояния кэша после перезагрузки узла, что повышает надежность.

4 Высокая производительность. Redis оптимизирован для работы в памяти и обеспечивает минимальные задержки при доступе к данным.

Комбинация PostgreSQL для постоянного хранения данных и Redis для высокоскоростного кэширования является стандартом для построения высокопроизводительных и отказоустойчивых систем. Это позволяет обеспечить необходимый баланс между надежностью, масштабируемостью и производительностью для всех операций микросервисного приложения агрегатора такси.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

Функциональное проектирование направлено на формирование внутренней структуры программной системы, чтобы она надежно и эффективно выполняла все поставленные перед ней задачи. В данном разделе приводится описание основных компонентов инфраструктуры для микросервисного приложения агрегатора такси, включая их внутренние модули, классы, способы взаимодействия и потоки данных между ними.

Архитектура системы реализована в виде набора независимых микросервисов, взаимодействующих между собой через REST API и брокер сообщений Kafka. Логические блоки, выделенные в системном проектировании, представляют собой микросервис или набор микросервисов. Каждый микросервис и инфраструктурный контейнер в системе выделен в отдельный модуль, логически разграничивающий ответственность и упрощающий сопровождение и масштабируемость системы.

Условно разделить блоки можно следующим образом:

1 Уровень бизнес-логики. Представляет из себя уровень, состоящий из блока базовых микросервисов, которые отвечают за базовую бизнес-логику. Включает в себя модули: сервис пассажиров, сервис водителей, сервис рейтинга, сервис поездок и сервис уведомлений.

2 Уровень инфраструктуры. Состоит из блоков: API-шлюза, аутентификации и авторизации, обнаружения базовых микросервисов, брокера сообщений, мониторинга и наблюдаемости, баз данных и кэширования.

В последующих подразделах представлено детальное описание каждого блока и модуля, их ключевых классов и механизмов взаимодействия, используемых для реализации бизнес-процессов агрегатора такси.

Диаграмма классов представлена на чертеже ГУИР.400201.047 РР.1, на котором отображены разработанные классы микросервисов с базовой бизнес-логикой. Диаграмма развертывания представлена на чертеже ГУИР.400201.047 РР.2.

3.1 Блок API-шлюза

Блок API-шлюза (gateway-service) является единой точкой входа для всех клиентских запросов в приложении. Блок построен на базе фреймворка Spring Cloud Gateway, который обеспечивает маршрутизацию запросов к соответствующим микросервисам, динамическое обнаружение сервисов через Eureka (Service Discovery) и централизованную настройку безопасности и CORS-политик.

3.1.1 Конфигурация маршрутизации.

Маршрутизация запросов определяется в конфигурационном файле `application.yml`. Система использует гибкое и динамическое управление маршрутизацией через предикаты для определения путей запроса и фильтров

для трансформации URL-адресов.

В таблице 3.1 представлены сервисы и паттерны маршрутизации:

Таблица 3.1 – Маршрутизация к сервисам инфраструктуры

Сервис	Паттерн маршрутизации
Сервис водителей	/api/v1/cars/**, /api/v1/drivers/**
Сервис пассажиров	/api/v1/passengers/**
Сервис рейтингов	/api/v1/passengers-ratings/**, /api/v1/drivers-ratings/**
Сервис поездок	/api/v1/rides/**
Сервис регистрации	/api/v1/cab-aggregator/**

Для корректной работы с микросервисами используется фильтр `RewritePath`, который удаляет префикс микросервиса из пути запроса, передавая на целевой сервис «очищенный» путь, соответствующий его внутренним контроллерам.

3.1.2 Обеспечение кросс-доменного взаимодействия (CORS).

Блок содержит централизованную настройку CORS, которая позволяет frontend-приложениям взаимодействовать с API-шлюзом из различных доменов. Конфигурация `[/**]` разрешает доступ ко всем методам (`allowedMethods: "*"`) и заголовкам (`allowedHeaders: "*"`) с любых источников (`allowedOrigins: "*"`), что обеспечивает гибкость интеграции на этапе разработки и тестирования.

3.1.3 Для интеграции с OpenAPI (Swagger) в модуле `application.yaml` файле шлюза настроен `springdoc-openapi`. Шлюз агрегирует документацию всех подключенных микросервисов.

Параметр `urls` в секции `swagger-ui` содержит перечень эндпоинтов `/v3/api-docs` для каждого сервиса, что позволяет пользователю переключаться между документацией Driver Service, Passenger Service, Rating Service и др. в едином интерфейсе по адресу `/swagger-ui`

3.1.4 Инфраструктурные настройки также предусмотрены в API-шлюзе, для поддержания наблюдаемости системы. Шлюз настроен на экспорт метрик в Prometheus и трассировку запросов через Zipkin/Tempo.

Для Eureka Client настроен `defaultZone` для регистрации шлюза в системе обнаружения сервисов, что позволяет ему динамически находить URI микросервисов по их именам.

Для Prometheus/Zipkin настроен параметр, который экспортирует данные о задержках и количестве запросов, а также пробрасывает заголовки трассировки для мониторинга полного пути запроса от шлюза до конкретного сервиса.

3.1.5 Параметры контейнеризации и развертывания.

Развертывание API-шлюза осуществляется с использованием технологии контейнеризации Docker. Конфигурация шлюза в среде оркестрации описывается через файл `docker-compose.yml`. Ниже представлены ключевые параметры конфигурации сервиса `gateway-service`:

Таблица 3.2 – Параметры контейнера `gateway-service`

Параметр	Значение	Описание
<code>container_name</code>	<code>gateway-service</code>	Наименование контейнера
<code>ports</code>	<code>8080:8080</code>	Внешний и внутренний порт системы
<code>networks</code>	<code>cab-app-network</code>	Наименование общей сети
<code>environment</code>	<code>LOKI: http://loki:3100,</code> <code>TEMPO_ZIPKIN_URL: http://tempo:9411</code>	Переменные окружения для интеграции с другими контейнерами (Loki и Tempo)
<code>healthcheck</code>	<code>curl f http://gateway service:8080/actuator/health</code>	Скрипт для проверки состояния контейнера

3.2 Блок аутентификации и авторизации

Блок аутентификации и авторизации обеспечивает безопасность инфраструктуры для приложения агрегатора такси. Он включает в себя модуль сервиса регистрации (`registration-service`) и инфраструктурные модули.

Выделение отдельного модуля сервиса регистрации для проксирования запросов в Keycloak облегчает взаимодействия со сложным API Keycloak, с его множеством параметров запросов. Выделение отдельного инфраструктурного модуля Keycloak позволяет обеспечить централизованную защиту ресурсов.

3.2.1 Модуль сервиса регистрации.

Модуль сервиса регистрации предназначен для управления учетными записями пользователей, обеспечения входа в систему и интеграции с Keycloak для проверки подлинности.

Класс `UserManagementServiceImpl` отвечает за реализацию бизнес-логики регистрации и аутентификации.

При инициализации экземпляра класса принимает следующие

переменные:

- `keycloak` (тип `Keycloak`) – административный клиент Keycloak для управления учетными записями;
- `restTemplate` (тип `RestTemplate`) – клиент для выполнения HTTP-запросов к Keycloak;
- `keycloakProperties` (тип `KeycloakProperties`) – параметры конфигурации сервера аутентификации;
- `driverFeignClient` и `passengerFeignClient` – клиенты для синхронизации создания профилей в соответствующих сервисах;
- `messageSource` (тип `MessageSource`) – сервис интернационализации.

Методы класса:

1 `signUp` выполняет регистрацию пользователя: создает учетную запись в Keycloak, назначает роль (`PASSENGER` или `DRIVER`) и вызывает соответствующий сервис (`driver` или `passenger`) для создания профиля в базе данных. В случае ошибки обеспечивает откат (удаление учетной записи из Keycloak).

2 `signIn` реализует аутентификацию пользователя по логину и паролю. Возвращает `UserKeycloakTokenResponseDto` с JWT-токенами.

3 `signInAsAdmin` реализует аутентификацию для административных целей через `client_credentials`, возвращая административный токен доступа.

Класс-контроллер `UserManagementController` отвечает за обработку входящих HTTP-запросов. Реализует интерфейс `UserManagementOperations` для поддержки OpenAPI и использует `UserManagementService` для выполнения операций.

Методы контроллера:

1 `signIn` обрабатывает POST-запросы по пути `/api/v1/cab-aggregator/signin`. Принимает объект `SignInDto`, содержащий email и пароль, и возвращает токен доступа.

2 `signUp` обрабатывает POST-запросы по пути `/api/v1/cab-aggregator/signup`. Принимает `SignUpDto` с персональными данными пользователя и ролью, выполняя процесс регистрации.

3 `signInAsAdmin` обрабатывает POST-запросы по пути `/api/v1/cab-aggregator/signin/admin`. Используется для аутентификации администратора системы с помощью `SignInAdminDto`.

3.2.2 Инфраструктурный модуль Keycloak.

Для управления доступом используется Keycloak в контейнеризированной среде. Использование open-source решения в качестве провайдера идентификации на локальных серверах соответствует всем условиям безопасности, которые необходимы в крупных приложениях корпоративного уровня. Это обеспечивает надежную и контролируемую среду аутентификации. Конфигурационные параметры контейнеризованного модуля Keycloak и его базы данных в `docker-compose.yml` представлены таблице 3.3.

Таблица 3.3 – Параметры контейнера keycloak

Параметр	Значение	Описание
container_name	keycloak	Наименование контейнера
ports	8090:8090, 8443:8443	Порты для HTTP и HTTPS взаимодействия
networks	cab-app-network	Наименование общей сети
volumes	/keycloak/realms:/opt/bitnami/./data/import	Монтирование директории с настройками Realm
depends_on	keycloak_db	Ожидание готовности базы данных Keycloak

3.2.3 Параметры развертывания и конфигурации для registration-service.

Для обеспечения гибкости настройки сервиса, возможности изменения конфигурации без дополнительной сборки Docker-образа и соблюдения принципов обеспечения безопасности данных, все ключевые параметры подключения вынесены в переменные среды.

В таблице 3.4 представлены переменные среды для запуска registration-service.

Таблица 3.4 – Переменные среды для настройки registration-service

Параметр	Значение
EUREKA_SERVER_URL	Адрес сервера регистрации микросервисов (Eureka) для взаимодействия с Discovery Service
CLIENT_ID_FOR_KC_ADMIN_CLIENT	Идентификатор клиента, используемый для авторизации сервиса в Admin API Keycloak
CLIENT_SECRET_FOR_KC_ADMIN_CLIENT	Секретный ключ, необходимый для аутентификации административного клиента
USERNAME_FOR_KC_ADMIN_CLIENT	Имя учетной записи администратора (service-account) в Keycloak
PASSWORD_FOR_KC_ADMIN_CLIENT	Пароль администратора, используемый для получения токенов доступа
SERVER_URL_FOR_KC_ADMIN_CLIENT	Базовый URL-адрес сервера Keycloak, к которому обращается сервис
CLIENT_ID_FOR_AUTH_KC	Идентификатор клиента для выполнения операций аутентификации пользователей

3.3 Блок обнаружения сервисов

Блок сервиса обнаружения (discovery-service) является центральным узлом микросервисной инфраструктуры, реализующим паттерн Service Registry. Он обеспечивает динамическое обнаружение микросервисов в системе, позволяя компонентам находить друг друга по логическим именам, а не по статическим IP-адресам. Блок построен на основе проекта Netflix Eureka Server.

3.3.1 Конфигурация блока.

Конфигурация discovery-service определяется в файле application.yml.

Основные параметры конфигурации:

- register-with-eureka: false – указывает, что данный сервис является сервером и не должен регистрироваться как клиент в других реестрах;
- fetch-registry: false – отключает попытки получения реестра от других серверов;
- metadataMap – содержит метаданные для системы автоматического сбора метрик Prometheus, позволяя системе мониторинга автоматически обнаруживать новые микросервисы.

3.3.2 Параметры контейнеризации и развертывания.

Развертывание сервиса обнаружения осуществляется с использованием технологии контейнеризации Docker. Конфигурация в среде оркестрации описывается через файл docker-compose.yml. Ниже представлены ключевые параметры конфигурации сервиса discovery-service:

Таблица 3.5 – Параметры контейнера discovery-service

Параметр	Значение	Описание
container_name	discovery-service	Наименование контейнера
ports	8761:8761	Внешний и внутренний порт системы
networks	cab-app-network	Наименование общей сети
environment	LOKI: http://loki:3100, TEMPO_ZIPKIN_URL: http://tempo:9411	Переменные окружения для интеграции с системами мониторинга
healthcheck	curl f http://discovery service:8761/actuator/health	Проверка состояния

3.4 Блок базовых микросервисов

Блок базовых микросервисов отвечает за реализацию основной бизнес-логики приложения агрегатора такси, управляя различными аспектами взаимодействия пользователей, поездок, рейтингов и уведомлений.

Блок включает в себя пять основных модулей: сервис пассажиров, сервис водителей, сервис рейтинга, сервис поездок и сервис уведомлений. Данные модули имеют слабую связность и предназначены для наполнения базовой бизнес-логикой.

3.4.1 Модуль сервиса пассажиров.

Модуль сервиса пассажиров (`passenger-service`) предназначен для управления учетными данными пользователей, имеющих роль пассажира. Он обеспечивает полный жизненный цикл работы с профилем пассажира: создание, обновление, получение данных и удаление, а также управление пользовательскими аватарами. Модуль интегрирован с блоком брокера сообщений Kafka для обработки событий обновления рейтинга, поступающих из службы рейтингов.

Класс `PassengerServiceImpl` отвечает за реализацию бизнес-логики управления пассажирами.

При инициализации экземпляра класса принимает следующие переменные

- `passengerRepository` (тип `PassengerRepository`) – репозиторий для выполнения операций с БД PostgreSQL;
- `passengerMapper` (тип `PassengerMapper`) – маппер для преобразования между DTO и сущностями;
- `listContainerMapper` (тип `ListContainerMapper`) – утилитарный маппер для формирования ответов с пагинацией;
- `messageSource` (тип `MessageSource`) – сервис интернационализации сообщений.

Методы класса:

1 `createPassenger` выполняет регистрацию нового пассажира, проверяя уникальность телефона и email. Использует флаг `deleted` для корректной обработки восстановления записей.

2 `updatePassengerById` обновляет данные существующего пассажира. Выполняет проверку на дубликаты данных перед сохранением.

3 `safeDeletePassengerById` реализует логическое удаление записи путем установки флага `deleted = true`.

4 `getPagePassengers` возвращает постраничный список всех активных пассажиров.

5 `getPassengerById` метод получения детальной информации о пассажире по его идентификатору.

Класс `PassengerServiceImpl` является реализацией интерфейса `PassengerService`, который в дальнейшем внедряется в экземпляр класса

REST-контроллера.

Класс `AvatarServiceImpl` отвечает за работу с медиаконтентом (аватарами) пассажиров в MinIO.

Класс имеет следующие переменные экземпляра:

- `minioClient` (тип `MinioClient`) – клиент для взаимодействия с S3-совместимым хранилищем;
- `passengerRepository` (тип `PassengerRepository`) – используется для подтверждения существования пассажира перед операциями с файлом;
- `bucketName`, `allowedTypes` – параметры конфигурации хранилища, загружаемые из свойств приложения.

Методы класса:

1 `uploadAvatar` принимает файл, проверяет его тип на соответствие списку разрешенных (JPEG, PNG) и загружает в MinIO с именем, привязанным к ID пассажира.

2 `getAvatar` извлекает поток данных аватара из MinIO.

3 `deleteAvatar` удаляет объект из хранилища, проверяя его наличие перед операцией.

Класс `AvatarServiceImpl` является реализацией интерфейса `AvatarService`, который в дальнейшем внедряется в экземпляр класса REST-контроллера с соответствующими операциями.

Класс `AverageRatingListener` реализует обработку сообщений из брокера Kafka.

Данный класс содержит следующие переменные экземпляра:

- `passengerRepository` (тип `PassengerRepository`) – для поиска пассажиров;
- `passengerMapper` (тип `PassengerMapper`) – для обновления рейтинга в сущности;
- `messageSource` (тип `MessageSource`) – для обработки исключений.

Метод `listenToDriverAverageRating` – аннотирован `@KafkaListener`, принимает сообщение `AverageRatingResponseDto`. При получении обновляет средний рейтинг пассажира в базе данных.

Класс `ValidateAccessToResourcesAspect` обеспечивает безопасность доступа к ресурсам

Переменная экземпляра `passengerService` (тип `PassengerService`) используется для верификации владельца профиля.

Метод `hasPermission` выполняет проверку прав доступа перед вызовом защищенных методов. Если пользователь не является администратором, проверяет совпадение email из JWT-токена с email владельца запрашиваемого ресурса.

Данный класс внедряется с помощью Spring и использует концепцию AOP, при которой в каждый метод REST-контроллера добавляется дополнительная логика.

Интерфейс `PassengerRepository` наследуется от `JpaRepository` (класс библиотеки Spring Data) и предоставляет следующие методы для

взаимодействия с базой данных:

- 1 `findByIdAndDeletedIsFalse` – поиск активного пассажира.
- 2 `existsByEmailAndDeletedIsFalse` – проверка уникальности email.
- 3 `findAllByDeletedIsFalse` – получение активных записей с поддержкой пагинации.

Данные методы взаимодействуют с сущностью `Hibernate Passenger`, которая содержит в себе следующие переменные экземпляра:

- `id` (тип `Long`) – уникальный идентификатор объекта;
- `firstName` (тип `String`) – имя пассажира;
- `lastName` (тип `String`) – фамилия пассажира;
- `email` (тип `String`) – адрес электронной почты пассажира;
- `phone` (тип `String`) – номер телефона пассажира;
- `averageRating` (тип `Double`) – средний рейтинг пассажира;
- `deleted` (тип `boolean`) – флаг наличия записи в базе данных.

Данный класс используется в сервисных классах микросервиса пассажиров.

Класс-контроллер `PassengerController` отвечает за обработку HTTP-запросов, связанных с базовыми операциями над профилями пассажиров.

Класс реализует интерфейс `PassengerOperations`, используемый для OpenAPI документации и внедряет зависимость `PassengerService` для выполнения бизнес-логики.

Класс-контроллер имеет следующие методы:

1 `createPassenger` обрабатывает POST-запросы для регистрации новых пассажиров. Ограничен ролью `ROLE_ADMIN`.

2 `getPagePassengers` обрабатывает GET-запросы для получения списка пассажиров с поддержкой пагинации.

3 `safeDeletePassenger` обрабатывает DELETE-запросы для логического удаления пассажира. Использует аспект `ValidateAccessToResources` для проверки прав владельца профиля или администратора.

4 `updatePassengerById` обрабатывает PUT-запросы для изменения профиля. Также защищен аспектом проверки доступа.

5 `getPassengerById` обрабатывает GET-запросы для получения данных конкретного пассажира по его ID.

Класс-контроллер `AvatarController` отвечает за обработку HTTP-запросов, связанных с загрузкой, получением и удалением аватаров пассажиров.

Класс реализует интерфейс `AvatarOperations`, используемый для OpenAPI документации и внедряет зависимость `PassengerService` для выполнения бизнес-логики.

Класс-контроллер имеет следующие методы:

1 `uploadAvatar` обрабатывает POST-запросы с `MultipartFile`. Реализует логику проверки прав доступа (через `@PreAuthorize`) и валидацию принадлежности ресурса владельцу.

2 `getAvatar` обрабатывает GET-запросы, возвращая `InputStreamResource` для отображения изображения.

3 `deleteAvatar` обрабатывает DELETE-запросы для удаления аватара из хранилища. Ограничен доступом для ADMIN или владельца профиля.

3.4.2 Модуль сервиса водителей.

Модуль сервиса водителей (`driver-service`) предназначен для управления учетными данными водителей, их транспортными средствами, а также для обработки медиаконтента. Модуль обеспечивает жизненный цикл профиля водителя, включая создание, обновление, получение данных и безопасное удаление. Также модуль интегрирован с брокером сообщений Kafka для получения обновлений рейтинга и отправки статистических отчетов в службу уведомлений.

Класс `DriverServiceImpl` отвечает за реализацию бизнес-логики управления водителями.

При инициализации экземпляра класса принимает следующие переменные:

- `driverRepository` (тип `DriverRepository`) – репозиторий для выполнения операций с БД PostgreSQL;
- `driverMapper` (тип `DriverMapper`) – маппер для преобразования между DTO и сущностями;
- `listContainerMapper` (тип `ListContainerMapper`) – утилитарный маппер для пагинации;
- `messageSource` (тип `MessageSource`) – сервис интернационализации сообщений.

Методы класса:

1 `createDriver` выполняет регистрацию нового водителя, проверяя уникальность телефона и email, а также возможность восстановления ранее удаленной записи.

2 `getPageDrivers` возвращает постраничный список активных водителей.

3 `safeDeleteDriverByDriverId` реализует логическое удаление записи установкой флага `deleted = true`.

4 `updateDriverById` обновляет данные водителя с проверкой дубликатов email/телефона.

5 `getDriverWithCars` предоставляет полную информацию о водителе, включая агрегированный список всех связанных с ним автомобилей.

Класс `DriverServiceImpl` является реализацией интерфейса `DriverService`, который в дальнейшем внедряется в экземпляр класса REST-контроллера.

Класс `CarServiceImpl` отвечает за управление парком автомобилей, привязанных к водителям.

Переменные экземпляра класса:

- `carRepository` (тип `CarRepository`) – репозиторий для управления

автомобилями;

- `driverRepository` (тип `DriverRepository`) – репозиторий для привязки автомобиля к конкретному водителю;

- `carMapper` (тип `CarMapper`) – маппер для работы с DTO автомобилей.

Методы экземпляра класса:

- 1 `createCar` регистрирует автомобиль и привязывает его к существующему водителю.

- 2 `updateCarByCarIdAndDriverId` изменяет характеристики автомобиля.

- 3 `safeDeleteCarByCarId` выполняет логическое удаление автомобиля.

- 4 `getCarById` метод получения детальной информации об автомобиле.

Класс `CarServiceImpl` является реализацией интерфейса `CarService`, который в дальнейшем внедряется в экземпляр класса REST-контроллера.

Класс `AvatarServiceImpl` отвечает за работу с аватарами водителей в `MinIO`.

Переменные экземпляра аналогичны реализации в модуле пассажиров: `minioClient`, `driverRepository`, `bucketName`, `allowedTypes`.

Методы класса:

- 1 `uploadAvatar` принимает изображение, проверяет расширение и сохраняет в объектное хранилище.

- 2 `getAvatar` получает поток данных для отображения аватара.

- 3 `deleteAvatar` удаляет файл из `MinIO`.

Класс `AvatarServiceImpl` является реализацией интерфейса `AvatarService`, который в дальнейшем внедряется в экземпляр класса REST-контроллера.

Класс `StatisticReportServiceImpl` реализует логику отправки периодических отчетов.

Экземпляр класса принимает следующие переменные:

- `rideFeignClient` (тип `RightFeignClient`) – репозиторий для управления автомобилями;

- `driverRepository` (тип `DriverRepository`) – репозиторий для привязки автомобиля к конкретному водителю;

- `driverMapper` (тип `CarMapper`) – маппер для работы с DTO автомобилей.

Метод `scheduledSenderStatisticToNotificationService` имеет аннотацию `@Scheduled` и запускается по расписанию (cron-выражение). Он собирает данные о поездках для всех водителей и отправляет их в `Kafka` для дальнейшей обработки сервисом уведомлений.

Класс `AverageRatingListener` реализует обработку входящих сообщений `Kafka`.

Метод `listenToDriverAverageRating` принимает сообщение `AverageRatingResponseDto`, находит водителя по `refUserId` и обновляет его рейтинг в БД.

Класс `ValidateAccessToResourcesAspect` обеспечивает безопасность

доступа к ресурсам аналогично пункту 3.1.4

Интерфейсы репозитория `DriverRepository` и `CarRepository` наследуются от `JpaRepository` (библиотека Spring Data) и предоставляют методы для взаимодействия с базой данных.

Интерфейс `DriverRepository` содержит методы:

1 `findByIdAndDeletedIsFalse` осуществляет поиск активного водителя по ID с использованием графа сущностей `cars` для предотвращения N+1 проблемы при выборке списка машин.

2 `existsByEmailAndDeletedIsFalse` выполняет проверку уникальности email среди активных записей.

3 `existsByPhoneAndDeletedIsFalse` выполняет проверку уникальности телефона среди активных записей.

4 `findAllByDeletedIsFalse` выполняет получение списка активных водителей с поддержкой пагинации.

Данные методы взаимодействуют с сущностью `Hibernate Driver`, которая содержит:

- `id` (тип `Long`) – уникальный идентификатор объекта;
- `firstName` (тип `String`) – имя водителя;
- `lastName` (тип `String`) – фамилия водителя;
- `email` (тип `String`) – адрес электронной почты;
- `phone` (тип `String`) – номер телефона;
- `gender` (тип `String`) – пол водителя;
- `averageRating` (тип `Double`) – средний рейтинг;
- `deleted` (тип `boolean`) – флаг логического удаления;
- `cars` (тип `List<Car>`) – список привязанных к водителю транспортных средств.

Интерфейс `CarRepository` содержит методы:

1 `existsByNumberAndDeletedIsFalse` выполняет проверка, существует ли уже активная машина с таким номером.

2 `findByIdAndDeletedIsFalse` осуществляет поиск активного автомобиля по ID.

Данные методы взаимодействуют с сущностью `Car`, которая содержит:

- `id` (тип `Long`) – уникальный идентификатор объекта;
- `brand` (тип `String`) – марка автомобиля;
- `color` (тип `String`) – цвет автомобиля;
- `number` (тип `String`) – лицензионный номер автомобиля;
- `model` (тип `String`) – модель автомобиля;
- `year` (тип `Integer`) – год выпуска;
- `deleted` (тип `boolean`) – флаг логического удаления;
- `driver` (тип `Driver`) – связь @ManyToOne, указывающая на владельца автомобиля.

Данные интерфейсы используются в соответствующих сервисных методах для взаимодействия с базой данных.

Класс-контроллер `DriverController` отвечает за обработку HTTP-

запросов, связанных с управлением профилями водителей.

Класс реализует интерфейс `DriverOperations`, используемый для OpenAPI документации и внедряет зависимость `DriverService` для выполнения бизнес-логики.

Класс контроллер содержит следующие методы:

1 `createDriver` обрабатывает POST-запрос для создания нового профиля водителя. Доступ ограничен ролью `ROLE_ADMIN`.

2 `getPageDrivers` обрабатывает GET-запрос для получения постраничного списка водителей.

3 `safeDeleteDriver` обрабатывает DELETE-запрос для логического удаления водителя. Использует аспект `@ValidateAccessToResources` для контроля доступа.

4 `updateDriverById` обрабатывает PUT-запрос для редактирования данных водителя. Защищен аспектом проверки доступа для `ADMIN` или самого водителя.

5 `getDriverById` обрабатывает GET-запрос для получения информации о водителе.

6 `getDriverWithCars` обрабатывает GET-запрос для получения детального профиля водителя вместе с перечнем его транспортных средств.

Класс-контроллер `CarController` отвечает за обработку HTTP-запросов, связанных с управлением профилями водителей.

Класс реализует интерфейс `CarOperations`, используемый для OpenAPI документации и внедряет зависимость `CarService`.

Методы класса:

1 `createCar` обрабатывает POST-запрос для привязки нового автомобиля к водителю. Доступ только для `ADMIN`.

2 `updateCarByCarIdAndDriverId` обрабатывает PUT-запрос для изменения параметров автомобиля. Доступ только для `ADMIN`.

3 `safeDeleteCarById` обрабатывает DELETE-запрос для логического удаления записи об автомобиле. Доступ только для `ADMIN`.

4 `getCarById` обрабатывает GET-запрос для получения характеристик конкретного автомобиля по ID.

Класс-контроллер `AvatarController` отвечает за обработку HTTP-запросов, связанных с управлением медиаконтентом водителей.

Класс реализует интерфейс `AvatarOperations`, используемый для OpenAPI документации и внедряет зависимость `AvatarService`.

Содержит следующие методы:

1 `uploadAvatar` обрабатывает POST-запросы с `MultipartFile` для загрузки аватара. Проверяет права доступа через `@PreAuthorize`.

2 `getAvatar` обрабатывает GET-запросы для потоковой передачи изображения аватара.

3.4.3 Модуль сервиса рейтинга.

Модуль сервиса рейтинга (`rating-service`) предназначен для

аккумуляции оценок пользователей, их хранения и расчета средних значений. Он обеспечивает прозрачность взаимодействия между участниками поездок, позволяя выставлять оценки и оставлять комментарии. Модуль является высоконагруженным, поэтому использует кэширование данных в Redis и асинхронное взаимодействие через Kafka для информирования других сервисов (например, driver-service и passenger-service) об изменении рейтинга пользователей.

Модуль использует абстрактный класс `AbstractRatingService`, который содержит общие методы для работы с рейтингами, что обеспечивает соблюдение принципа DRY (Don't Repeat Yourself).

Классы `DriverRatingService` и `PassengerRatingService` наследуют функционал от абстрактного родителя и специализируются на работе с соответствующими репозиториями.

Экземпляры сервисов принимают:

- `repository` (тип `CommonRatingRepository<T>`) – репозиторий для доступа к сущностям рейтинга;
- `ratingMapper` (тип `BaseRatingMapper<T>`) – маппер для преобразования DTO;
- `listContainerMapper` (тип `ListContainerMapper`) – утилитарный маппер для пагинации;
- `rideFeignClient` (тип `RideFeignClient`) – клиент для взаимодействия с `rides-service` для валидации поездок;
- `averageRatingSender` (тип `AverageRatingSender`) – компонент для публикации событий рейтинга в Kafka.

Методы классов (реализованные в `AbstractRatingService`):

1 `createRating` создает новую запись о рейтинге. Метод запрашивает информацию о поездке через `rideFeignClient`, проверяет отсутствие дубликатов по `rideId` и сохраняет оценку.

2 `updateRatingById` обновляет содержимое комментария и оценку.

3 `getRating` возвращает детализированную информацию о конкретном рейтинге.

4 `getRatingsByRefUserId` возвращает постраничный список оценок, полученных конкретным пользователем (водителем или пассажиром).

5 `safeDeleteRating` выполняет «мягкое» удаление оценки (`deleted = true`), предотвращая ее отображение в расчетах.

6 `getAverageRating` вычисляет среднее значение рейтинга для пользователя и инициирует асинхронную отправку обновленных данных через Kafka.

В качестве репозитория и сущностей данных используется абстрактный общий класс `CommonRatingRepository<T>` с соответствующими сущностями `DriverRating` и `PassengerRating` с общим суперклассом `Rating`.

Интерфейс `CommonRatingRepository<T>` предоставляет общие методы для доступа к данным:

1 `existsByRideIdAndDeletedIsFalse` выполняет проверку, выставлена

ли уже оценка за данную поездку.

2 `findByIdAndDeletedIsFalse` осуществляет поиск активного рейтинга.

3 `findAllByRefUserIdAndDeletedIsFalse` выполняет выборку оценок конкретного пользователя.

4 `getAverageRatingByRefUserId` выполняет расчет среднего рейтинга.

Сущность `Rating (MappedSuperclass)` содержит общие поля:

- `id` (тип `Long`) – уникальный идентификатор;
- `comment` (тип `String`) – текст отзыва;
- `rating` (тип `Integer`) – оценка (от 1 до 5);
- `rideId` (тип `Long`) – идентификатор связанной поездки;
- `refUserId` (тип `Long`) – идентификатор оцениваемого пользователя;
- `deleted` (тип `boolean`) – флаг логического удаления.

Классы-наследники `DriverRating` и `PassengerRating` являются самостоятельными сущностями `@Entity` с собственными генераторами последовательностей ID.

Для взаимодействия с сервисов по REST API используются классы-контроллеры `DriverRatingController` и `PassengerRatingController`.

`DriverRatingController` реализует `DriverRatingOperations`.

Методы:

1 `createDriverRating` обрабатывает POST-запрос для создания оценки водителю. Доступно для ADMIN и DRIVER.

2 `updateDriverRating` обрабатывает PUT-запрос для изменения оценки. Доступ только для ADMIN.

3 `getDriverRating` обрабатывает GET-запрос для получения информации об оценке.

4 `getDriverRatingsByRefUserId` обрабатывает GET-запрос для получения истории оценок водителя.

5 `deleteDriverRating` обрабатывает DELETE-запрос для логического удаления оценки.

6 `averageDriverRating` обрабатывает GET-запрос для получения среднего рейтинга водителя.

`PassengerRatingController` реализует `PassengerRatingOperations`.

Методы аналогичны `DriverRatingController` и обеспечивают управление оценками пассажиров. В качестве ограничений доступа используется `ROLE_ADMIN` и `ROLE_PASSENGER`.

Для асинхронного взаимодействия отвечает класс `AverageRatingSender`, который публикует события в Kafka. Данный класс принимает переменную `kafkaAverageRatingTemplate` (тип `KafkaTemplate<String, AverageRatingResponseDto>`)

Методы класса `AverageRatingSender`:

1 `sendAverageRatingToDriver` отправляет актуальный средний рейтинг в топик `driver-average-rating-topic`.

2 `sendAverageRatingToPassenger` отправляет актуальный средний

рейтинг в топик `passenger-average-rating-topic`.

3.4.4 Модуль сервиса поездок.

Модуль сервиса поездок (`rides-service`) является ключевым узлом системы, координирующим взаимодействие между водителями и пассажирами. Он обеспечивает полный цикл обработки заказа: от инициализации (создания) до завершения или отмены поездки. Модуль отвечает за расчет стоимости, управление статусами заказа и агрегацию статистических данных для пользователей. Высокая доступность данных обеспечивается использованием Redis для кэширования, а корректность переходов состояний – встроенным механизмом валидации.

За бизнес-логику отвечает класс `RideServiceImpl` со следующими переменными экземпляра:

- `rideRepository` (тип `RideRepository`) – репозиторий для выполнения операций с БД PostgreSQL;
- `rideMapper` (тип `RideMapper`) – маппер для преобразования DTO в сущности и выполнения частичных обновлений;
- `listContainerMapper` (тип `ListContainerMapper`) – утилитарный маппер для пагинации;
- `messageSource` (тип `MessageSource`) – сервис интернационализации сообщений;
- `priceGenerator` (тип `RideServicePriceGenerator`) – компонент для генерации стоимости поездки;
- `rideServiceValidation` (тип `RideServiceValidation`) – компонент для проверки корректности переходов статусов и наличия пользователей.

Методы класса:

- 1 `createRide` создает новую поездку, присваивает ей начальный статус `CREATED`, генерирует стоимость и устанавливает текущую временную метку.
- 2 `changeRideStatus` выполняет обновление статуса поездки (например, `ACCEPTED`, `COMPLETED`), предварительно проверяя допустимость перехода состояний.
- 3 `updateRide` обновляет детали поездки (адреса, водителя, пассажира). Доступно только администратору.
- 4 `getRideById` возвращает информацию о конкретной поездке.
- 5 `getPageRides` возвращает страничный список всех существующих поездок.
- 6 `getRideStatisticForDriver` / `getRideStatisticForPassenger` – методы агрегации, вычисляющие среднюю стоимость поездок и формирующие список всех поездок пользователя для отчетов.

Класс `RideServiceValidation` обеспечивает целостность данных.

Переменные экземпляра класса `RideServiceValidation`:

- `messageSource` (тип `MessageSource`) – сервис для локализации сообщений об ошибках при нарушении правил перехода статусов;
- `passengerFeignClient` (тип `PassengerFeignClient`) – клиент для

проверки существования пассажира в системе;

– `driverFeignClient` (тип `DriverFeignClient`) – клиент для проверки существования водителя в системе.

Для проверки целостности данных в данном классе реализованы следующие методы:

1 `validateChangingRideStatus` содержит логику переходов состояний (нельзя перейти к `COMPLETED` из `CREATED` без промежуточных стадий).

2 `checkExistingPassengerOrDriver` выполняет удаленную проверку существования пользователей через Feign-клиенты.

Для взаимодействия с базой данных используется интерфейс `RideRepository`, который наследуется от `JpaRepository` и предоставляет методы доступа:

1 `findAllByDriverId` выполняет поиск всех поездок конкретного водителя с пагинацией.

2 `findAllByPassengerId` выполняет поиск всех поездок конкретного пассажира с пагинацией.

3 `findAllRidesForReportByDriverId` осуществляет JPQL-запросы, возвращающие список всех поездок (кроме отмененных) для формирования статистических отчетов.

Данный репозиторий оперирует сущностью `Ride` со следующими полям:

- `id` (тип `Long`) – уникальный идентификатор;
- `driverId` (тип `Long`) – идентификатор водителя;
- `passengerId` (тип `Long`) – идентификатор пассажира;
- `departureAddress` (тип `String`) – адрес отправления;
- `destinationAddress` (тип `String`) – адрес назначения;
- `rideStatus` (тип `RideStatus`) – текущий адрес поездки (enum);
- `orderDateTime` (тип `LocalDateTime`) – дата и время создания заказа;
- `cost` (тип `BigDecimal`) – стоимость поездки.

Для взаимодействия с сервером по HTTP используется класс-контроллер `RideController`, который реализует интерфейс `RideOperations`.

Класс `RideController` содержит следующие методы:

1 `createRide` обрабатывает POST-запрос для создания поездки. Доступен ADMIN и PASSENGER.

2 `changeRideStatus` обрабатывает PATCH-запрос для смены статуса. Доступен для всех ролей.

3 `updateRide` обрабатывает PUT-запрос для редактирования параметров поездки. Доступен только для ADMIN.

4 `getRideById` обрабатывает GET-запрос для получения информации о поездке.

5 `getPageRides` / `getPageRidesByDriverId` / `getPageRidesByPassengerId` обрабатывают GET-запросы для получения пагинированных списков поездок (общих или отфильтрованных по пользователю).

6 `getRideStatisticForPassenger` / `getRideStatisticForDriver`

обрабатывают GET-запросы для получения итоговой статистики по поездкам конкретного пользователя.

3.4.5 Модуль сервиса уведомлений.

Модуль сервиса уведомлений (notification-service) отвечает за автоматизированную рассылку отчетов о поездках пользователям по электронной почте. Модуль является потребителем событий Kafka, формирует PDF-отчеты и обеспечивает интеграцию с почтовыми протоколами.

Класс `NotificationServiceImpl` реализует логику формирования и отправки email-отчетов.

Переменные экземпляра:

- `mailSender` (тип `JavaMailSender`) – основной компонент для отправки электронных писем;
- `resourceLoader` (тип `ResourceLoader`) – вспомогательный компонент для загрузки HTML-шаблонов из ресурсов приложения

Основные методы:

1 `sendUserStatisticOnEmail` основной метод бизнес-логики. Получает данные статистики, формирует HTML-тело письма, преобразует его в PDF-документ с помощью `ITextRenderer` и отправляет письмо адресату с прикрепленным PDF-файлом.

2 `buildHtmlGreetingText` вспомогательный метод для динамической вставки имени пользователя в приветственный шаблон сообщения.

3 `buildReportAsStringForPdf` формирует HTML-структуру отчета, включая таблицу всех поездок пользователя и расчет средней стоимости, заменяя placeholders в шаблоне на реальные данные.

4 `generatePdf` выполняет отрисовку PDF-документа из подготовленной HTML-строки.

Для потребления сообщений из Kafka реализован класс `GeneralStatisticListener`, который принимает переменную `notificationService` (тип `NotificationService`) – сервис для обработки сформированных уведомлений.

Метод `listenToDriverAverageRating` аннотирован `@KafkaListener`, прослушивает топик `user-statistics-topic`. При поступлении сообщения `GeneralUserStatisticDto` (содержащего данные о поездках пользователя) вызывает сервис уведомлений для отправки отчета.

3.5 Блок брокера сообщений

Блок брокера сообщений обеспечивает асинхронное взаимодействие между микросервисами, гарантированную доставку сообщений и высокую пропускную способность системы для обмена событиями.

3.5.1 Параметры конфигурации кластера.

Кластер Apache Kafka развернут в изолированной сети `cab-app-network`

и состоит из трех узлов брокеров и одного узла Zookeeper, который координирует состояние кластера. Для мониторинга и управления топиками развернут веб-интерфейс kafka-ui.

В таблице 3.6 указаны основные параметры конфигурации узлов брокеров Kafka.

Таблица 3.6 – Основные конфигурационные параметры узлов брокеров Kafka

Параметр	Значение
KAFKA_CFG_BROKER_ID	Уникальный целочисленный идентификатор брокера (0, 1, 2) в пределах кластера
KAFKA_CFG_ZOOKEEPER_CONNECT	Адрес узла Zookeeper, отвечающего за хранение метаданных кластера и лидерство контроллеров
KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP	Определяет маппинг протоколов, что позволяет разделять трафик между узлами кластера и внешними клиентами
KAFKA_CFG_LISTENERS	Список портов и протоколов, на которых брокер ожидает подключения
KAFKA_CFG_ADVERTISED_LISTENERS	Адреса, которые брокер анонсирует клиентам. Это критически важный параметр для Docker, позволяющий клиентам извне сети обращаться к конкретному узлу
KAFKA_CFG_INTER_BROKER_LISTENER_NAME	Указывает протокол (INTERNAL), используемый для синхронизации данных (репликации) между брокерами.
ALLOW_PLAINTEXT_LISTENER	Разрешает передачу данных без шифрования

3.5.2 Параметры контейнеризации.

Развертывание кластера Apache Kafka осуществляется с помощью технологии контейнеризации Docker, что позволяет обеспечить идентичность среды выполнения на всех этапах разработки. В данной архитектуре брокер сообщений представлен тремя равноправными узлами, каждый из которых работает в изолированном контейнере. Такая конфигурация необходима для обеспечения высокой доступности системы: при выходе из строя одного из узлов кластер продолжает корректно обрабатывать сообщения за счет репликации данных между партициями.

Kafka является брокером сообщений, который хранит информацию с поступившими в него сообщениями (в отличие от RabbitMQ). Для предотвращения потери сообщений при перезапуске или удалении контейнеров используются именованные тома. Это гарантирует, что данные топиков и смещения потребителей сохраняются на физическом диске хост-машины. В таблице 3.7 представлены основные параметры конфигурации для контейнеров брокеров.

Таблица 3.7 – Параметры контейнеров брокеров

Параметр	Значение	Описание
container_name	kafka-0, kafka-1, kafka-2	Имена контейнеров для идентификации узлов
expose	9092	Порт, доступный другим контейнерам внутри сети
ports	29092-29094:29092-29094	Проброс портов на хост для доступа из IDE или средств мониторинга.
environment	KAFKA_CFG_BROKER_ID	Уникальный идентификатор узла (0, 1 или 2) для участия в распределенных процессах
volumes	kafka_x_data:/bitnami/kafka	Именованные тома для персистентного хранения логов сообщений.
healthcheck	kafka-topics.sh --list --bootstrap-server localhost:9092	Ожидание готовности базы данных Keycloak

3.6 Блок мониторинга и наблюдаемости

Блок мониторинга и наблюдаемости предназначен для отслеживания состояния микросервисов в реальном времени, анализа логов и выявления узких мест в производительности распределенной системы.

Блок реализован как набор независимых, но тесно интегрированных контейнеризированных компонентов: модуль сбора метрик (VictoriaMetrics), модуль агрегации логов (Loki), модуль распределенной трассировки (Tempo) и модуль визуализации (Grafana).

3.6.1 Модуль сбора метрик (VictoriaMetrics).

VictoriaMetrics выбрана в качестве базы данных временных рядов благодаря ее высокой производительности и экономичному потреблению ресурсов при хранении метрик. Модуль отвечает за сбор метрик от всех микросервисов. Параметры контейнера представлены в таблице 3.8.

Таблица 3.8 – Параметры контейнера VictoriaMetrics

Параметр	Значение	Описание
1	2	3
container_name	victoria-metrics	Имя контейнера для идентификации узла в кластере
volumes	./victoria-metrics/promscrape.yml:/...	Монтирование конфигурационного файла для сбора метрик

Продолжение таблицы 3.8

1	2	3
ports	8428:8428	Внешний и внутренний порт
command	promscrape.config=...	Запуск с указанием конфигурации поиска эндпоинтов микросервисов

3.6.2 Модуль агрегации логов (Loki)

Loki обеспечивает централизованный сбор логов из всех микросервисов. В таблице 3.9 представлены параметры контейнера.

Таблица 3.9 – Параметры контейнера Loki

Параметр	Значение	Описание
container_name	victoria-metrics	Имя контейнера, выступающего в роли сервера агрегации логов
ports	3100:3100	Внешний и внутренний порт API для отправки логов из микросервисов
volumes	./loki:/etc/loki	Именованные тома для хранения конфигурации и индекса логов
command	config.file=/etc/loki/loki-config.yml	Путь к основному файлу конфигурации Loki

3.6.3 Модуль распределенной трассировки (Tempo)

Tempo – система для хранения и визуализации трейсов (цепочек вызовов). При прохождении запроса через несколько микросервисов (например, Gateway → Driver Service → Ride Service), Tempo записывает время выполнения каждого этапа, позволяя выявить задержки в конкретном компоненте. В таблице 3.10 представлены параметры контейнера Tempo.

Таблица 3.10 – Параметры контейнера Tempo

Параметр	Значение	Описание
container_name	tempo	Имя контейнера для трассировки запросов
ports	14268, 3200, 4317, 9411	Порты для приема данных в разных протоколах (Jaeger, OTLP, Zipkin)
volumes	./tempo/tempo.yml:/etc/tempo.yml	Монтирование файла настроек процессора трассировок
command	config.file=/etc/tempo.yml	Команда запуска процесса Tempo с конфигурацией

3.6.4 Модуль визуализации (Grafana)

Grafana является единым графическим интерфейсом для анализа метрик, логов и трейсов. Она объединяет данные из VictoriaMetrics, Loki и Tempo,

предоставляя пользователю возможность видеть состояние всей инфраструктуры на единой панели мониторинга.

Конфигурация Grafana осуществляется декларативно с использованием механизма provisioning. Все настройки источников данных и визуальные представления метрик сохраняются в виде структурированных файлов в формате JSON. В таблице 3.11 представлены параметры контейнера Grafana.

Таблица 3.11 – Параметры контейнера Grafana

Параметр	Значение	Описание
container_name	grafana	Имя контейнера для визуализации данных
ports	3000:3000	Внешний порт доступа к web-интерфейсу Grafana
volumes	./grafana/provisioning:/etc/...	Монтирование настроек источников данных и дашбордов
environment	GF_AUTH_ANONYMOUS_ENABLED=true	Разрешение анонимного доступа к дашбордам для внутренних пользователей

3.7 Блок баз данных

Блок баз данных обеспечивает хранение структурированных данных для всех микросервисов приложения, реализуя принцип «Database per Service» для обеспечения слабой связанности, независимого масштабирования и отказоустойчивости.

Блок состоит из нескольких независимых экземпляров объектно-реляционной СУБД PostgreSQL, каждый из которых предназначен для конкретного микросервиса.

Архитектура хранения данных распределена по отдельным экземплярам СУБД:

- passengers_db хранит профили пассажиров и метаданные;
- drivers_db хранит профили водителей, данные об автомобилях и средние рейтинговые показатели;
- rides_db содержит историю всех транзакций поездок, логирование статусов и финансовую информацию;
- rating_db это специализированная БД для хранения оценок, отзывов и метаданных рейтингов.

3.7.1 Параметры контейнеризации баз данных.

Развертывание баз данных осуществляется в изолированной сети sub-app-network. Для обеспечения долговечности используются именованные тома, что предотвращает потерю данных при удалении или перезапуске контейнеров. Подход гарантирует сохранность критически важных данных и стабильность работы системы. Конфигурация приведена в таблице 3.12.

Таблица 3.12 – Параметры контейнеров баз данных

Параметр	Значение	Описание
container_name	passengers_db, drivers_db, rides_db, rating_db, keycloak_db	Наименование контейнеров СУБД
ports	5433-5436, 5440 : 5432	Проброс портов на хост для доступа из инструментов разработки
networks	cab-app-network	Изолированная сеть для внутреннего взаимодействия с сервисами
volumes	db_data:/var/lib/postgresql/data	Именованные тома для хранения файлов БД на хосте
environment	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	Параметры инициализации учетных данных и имени базы
healthcheck	pg_isready -U postgres	Скрипт проверки готовности PostgreSQL к приему соединений

3.8 Блок кэширования

Блок кэширования предназначен для снижения времени отклика системы и уменьшения нагрузки на реляционные базы данных путем хранения часто запрашиваемых данных в оперативной памяти.

Блок реализован на базе Redis, представляющего собой in-memory хранилище данных. В таблице 3.13 представлены параметры контейнера.

Таблица 3.13 – Параметры контейнера Redis

Параметр	Значение	Описание
container_name	redis-cache	Имя контейнера для идентификации в Docker
ports	6380:6379	Внешний и внутренний порты
networks	cab-app-network	Изолированная сеть для внутреннего взаимодействия с сервисами
volumes	redis_data:/data	Именованный том для сохранения дампа данных Redis
healthcheck	redis-cli ping	Команда проверки работоспособности сервиса

Блок кэширования обеспечивает высокую производительность и оперативность доступа к информации.

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

В данном разделе отражен процесс разработки программных модулей микросервисной системы. Так как полное приложение включает в себя несколько независимых сервисов, базовые CRUD-операции в которых имеют схожую структуру, в данном подразделе будут описаны только самые существенные компоненты, реализующие комплексную логику работы агрегатора такси. Остальные компоненты разработаны схожим образом.

Для наглядной демонстрации работы распределенной системы спроектирована общая диаграмма последовательностей, отражающая максимально полный сквозной бизнес-процесс: регистрация нового пользователя, аутентификация пользователя, загрузка пользователем персонального аватара, создание заявки на поездку, жизненный цикл поездки, публикация отзыва и оценки, асинхронный пересчет рейтинга и рассылка уведомлений на электронную почту. Диаграмма последовательностей представлена на чертеже ГУИР.400201.042 PP.3.

Листинг кода приведен в приложении А. Полные исходные коды находятся на диске в архиве «cab-aggregator-app.zip».

4.1 Реализация базовых микросервисов

4.1.1 Блок аутентификации и авторизации

Первым этапом является регистрация пользователя (водителя или пассажира). Блок `registration-service` выступает в роли оркестратора, синхронизируя данные между сервером авторизации Keycloak и базой данных микросервиса `driver-service`. Метод регистрации реализован в классе `UserManagementServiceImpl`.

Начальный этап работы метода заключается в подготовке данных и отправке запроса на создание пользователя в системе единого входа (SSO):

```
@Override
public void signUp(SignUpDto signUpDto) {
    UserRepresentation keycloakUser =
getUserRepresentation(signUpDto);
    RealmResource realmResource =
keycloak.realm(keycloakProperties.getRealm());
    UsersResource usersResource = realmResource.users();
    String adminClientAccessToken =
keycloak.tokenManager().getAccessTokenString();

    Response response =
Optional.ofNullable(usersResource.create(keycloakUser))
        .orElseThrow(() -> new
ServiceUnavailableException(messageSource.getMessage(
SERVICE_UNAVAILABLE_MESSAGE_KEY,
new Object[] {},
LocaleContextHolder.getLocale())));
```

Описание кода создания учетной записи:

1 Формирование объекта `UserRepresentation` для API Keycloak на основе входного объекта передачи данных `SignUpDto`.

2 Получение ресурсов сервера аутентификации (`RealmResource`, `UsersResource`) и генерация токена администратора `adminClientAccessToken` для безопасного межсервисного взаимодействия.

3 Отправка запроса `usersResource.create` к серверу Keycloak для создания учетной записи с обработкой ситуации недоступности сервиса через `Optional` и генерацией исключения `ServiceUnavailableException`.

После получения ответа от сервера Keycloak происходит проверка статуса выполнения операции и синхронизация с базами данных профилей через Feign-клиенты:

```
if (response.getStatus() == HttpStatus.CREATED.value()) {
    try {
        if (Objects.equals(signUpDto.role(),
PASSENGER_ROLE)) {
            passengerFeignClient.createPassenger(signUpDto,
LocaleContextHolder.getLocale().toLanguageTag(),
"Bearer " + adminClientAccessToken);
        } else {
            driverFeignClient.createDriver(signUpDto,
LocaleContextHolder.getLocale().toLanguageTag(),
"Bearer " + adminClientAccessToken);
        }
    } catch (Exception exception) {
        usersResource.delete(CreatedResponseUtil.getCreatedId(response));
        throw exception;
    }
} else {
    throw new KeycloakCreateUserException(
        new
ApiExceptionDto(HttpStatus.valueOf(response.getStatus()),
readResponseBody(response),
LocalDateTime.now()));}
```

Описание кода синхронизации и механизма отката:

1 Проверка HTTP-статуса ответа. Если статус 201 CREATED, инициируется синхронное взаимодействие с микросервисами профилей (`driverClient.createDriver()` или `passengerClient.createPassenger()`) с передачей сервисного токена администратора и информации о локале.

2 Реализация механизма распределенной транзакции через блок `try-catch`. Если микросервис профилей недоступен или возвращает ошибку валидации, выполняется компенсирующее действие – удаление только что созданного пользователя из Keycloak (`usersResource.delete`). Это строго

предотвращает появление «осиротевших» учетных записей.

3 В случае, если Keycloak вернул статус, отличный от успешного (например, конфликт email), генерируется пользовательское исключение `KeycloakCreateUserException` с извлечением сообщения об ошибке из тела ответа.

Заключительным этапом регистрации является присвоение пользователю соответствующей роли доступа:

```
RolesResource rolesResource = realmResource.roles();
RoleRepresentation role =
rolesResource.get(signUpDto.role()).toRepresentation();
UserResource userById =
usersResource.get(CreatedResponseUtil.getCreatedId(response));
userById.roles().realmLevel().add(List.of(role));}
```

Описание кода назначения ролей:

1 Извлечение объекта роли (`RoleRepresentation`) из ресурса ролей сервера Keycloak на основе данных, переданных клиентом при регистрации (`DRIVER` или `PASSENGER`).

2 Получение уникального идентификатора созданного пользователя с помощью утилиты `CreatedResponseUtil.getCreatedId()`.

3 Назначение пользователю соответствующей роли на уровне области (`realmLevel`), что позволяет в дальнейшем управлять доступом к защищенным конечным точкам микросервисов агрегатора такси.

4.1.2 Блок базовых микросервисов

Имея на руках JWT-токен, водитель загружает свое фото профиля. Данная логика реализована в микросервисе `driver-service` и `passenger-service` с использованием S3-совместимого хранилища MinIO. На примере сервиса для работы с водителями рассматривается метод `uploadAvatar` в классе `AvatarServiceImpl`:

```
public MinioFileInformation uploadAvatar(Long id,
MultipartFile file) {
    validateDriverExistence(id);
    validateFileType(file);
    InputStream inputStream = file.getInputStream();
    String newFileName =
AvatarServiceLiteralConstants.BASE_DRIVER_AVATAR_NAME + id;
    minioClient.putObject(
        PutObjectArgs.builder()
            .bucket(bucketName)
            .object(newFileName)
            .contentType(file.getContentType())
            .stream(inputStream,
inputStream.available(), -1)
            .headers(Map.of(Headers.CONTENT_DISPOSITION));
    return getImageAsMinioFileInformation(newFileName);}
```

Описание кода:

1 Проверка существования водителя в базе данных и валидация MIME-типа загружаемого файла (разрешены только image/jpeg и image/png).

2 Формирование уникального имени объекта на основе префикса и ID водителя.

3 Поточковая загрузка байтов в бакет MinIO с указанием метаданных и заголовков.

Далее водитель добавляет данные о своем автомобиле. Метод createCar в классе CarServiceImpl:

```
public CarDto createCar(Long driverId, CarDto carDto) {
    checkCarRestoreOption(carDto);
    checkCarExistsByNumber(carDto);
    Car carEntity = carMapper.toEntity(carDto);
    carEntity.setDeleted(false);
    Driver driverEntity =
        getDriverByIdAndDeletedIsFalse(driverId);
    carEntity.setDriver(driverEntity);
    driverRepository.save(driverEntity);
    carRepository.save(carEntity);
    return carMapper.toDto(carEntity);}
```

Описание кода:

1 Проверка уникальности номера автомобиля, чтобы избежать дубликатов в системе.

2 Преобразование DTO в JPA-сущность Car и установка связи @ManyToOne с сущностью Driver.

3 Сохранение изменений в PostgreSQL и автоматическое обновление кэша Redis благодаря аннотации @CachePut.

Следующие шаги – создание поездки пассажиром и поэтапное изменение ее статусов водителем. Вся координация происходит в микросервисе rides-service.

Метод создания поездки в классе RideServiceImpl:

```
public RideResponseDto createRide(RideRequestDto
rideRequestDto) {
    rideServiceValidation.checkExistingPassengerOrDriver(rideReq
uestDto);
    Ride ride = rideMapper.toEntity(rideRequestDto);
    ride.setRideStatus(RideStatus.CREATED);
    ride.setCost(priceGenerator.generateRandomCost());
    ride.setOrderDateTime(LocalDateTime.now());
    rideRepository.save(ride);
    return rideMapper.toDto(ride);}
```

Описание кода:

1 Валидация переданных идентификаторов: сервис делает синхронные HTTP-запросы через Feign к passenger-service и driver-service для

подтверждения существования пользователей.

2 Инициализация первичного статуса `CREATED`, времени заказа и расчет стоимости поездки.

3 Сохранение сущности в БД и возврат DTO.

После назначения заказа, водитель последовательно меняет статус поездки: принимает заказ, прибывает к пассажиру, начинает движение и завершает поездку. Метод изменения статуса:

```
public RideResponseDto changeRideStatus(Long rideId,
RideStatusRequestDto rideStatusRequestDto) {
    Ride ride = getRide(rideId);
    rideServiceValidation.validateChangingRideStatus(ride,
rideStatusRequestDto);
    rideMapper.partialUpdate(rideStatusRequestDto, ride);
    rideRepository.save(ride);
    return rideMapper.toDto(ride);}
```

Описание кода:

1 Метод жестко регламентирует переходы статусов. Невозможно пропустить логический этап (например, перейти из `CREATED` сразу в `COMPLETED`).

2 При нарушении логики генерируется написанное исключение `InvalidInputStatusException`.

3 При достижении терминального статуса (`COMPLETED` или `CANCELED`), результат кэшируется в Redis (`@CachePut`), что оптимизирует чтение истории поездок.

Далее пассажир оставляет отзыв водителю за завершённую поездку. Эта логика обрабатывается в `rating-service`.

Метод создания отзыва в классе `AbstractRatingService`:

```
public RatingResponseDto createRating(RatingRequestDto
ratingRequestDto) {
    Authentication authentication = getAuthentication();
    RideResponseDto rideResponseDto =
rideFeignClient.findRideById(
        ratingRequestDto.rideId(),
        LocaleContextHolder.getLocale().toLanguageTag(),
        "Bearer " + token.getToken().getTokenValue());
    checkRatingExistsByRideId(ratingRequestDto);
    T rating = ratingMapper.toRating(ratingRequestDto);
    setRefUserIdBasedOnUserType(rating, rideResponseDto);
    rating.setDeleted(false);
    repository.save(rating);
    return ratingMapper.toDto(rating, userType);}
```

Описание кода:

1 Синхронный запрос в `rides-service` для проверки существования указанной поездки.

2 Проверка бизнес-правила, исключающего дублирование отзывов для одной поездки (`checkRatingExistsByRideId`).

3 Сохранение отзыва в БД.

После обновления базы отзывов, `rating-service` пересчитывает средний рейтинг и асинхронно публикует его в брокер сообщений Apache Kafka. Метод публикации (Producer) в `AverageRatingSender`:

```
public void
sendAverageRatingToDriver(AverageRatingResponseDto
averageRatingResponseDto) {

    kafkaAverageRatingTemplate.send(KafkaConstants.DRIVER_TOPIC_NAME
    , averageRatingResponseDto)
        .whenComplete((result, exception) -> {
            if (exception != null) {
                log.error("Failed to send message: {}",
exception.getMessage());
            } else {
                log.info("Message sent successfully to
topic: {}", result.getRecordMetadata().topic());
            }
        });
}
```

Описание кода публикации сообщений (Producer):

1 Использование объекта `KafkaTemplate` для отправки сериализованного объекта передачи данных (`AverageRatingResponseDto`) в Kafka.

2 Применение неблокирующего метода `send()` в связке с функцией обратного вызова `whenComplete()`. Данный подход исключает задержки при ответе клиенту по HTTP, так как ожидание подтверждения от брокера происходит в фоновом потоке.

3 Логирование результатов отправки: в случае сетевых сбоев ошибка фиксируется в журнале сервера для последующего анализа, а при успехе записываются метаданные (название топика, партиция и смещение).

На стороне микросервиса водителей (`driver-service`) реализован получатель сообщений (Consumer) – класс `AverageRatingListener`. Для корректной обработки данных слушатель конфигурируется с помощью специализированной аннотации:

```
@KafkaListener(
    topics =
KafkaConstants.TOPIC_NAME_AVERAGE_RATING_FROM_RATING_SERVICE,
    groupId =
KafkaConstants.GROUP_ID_AVERAGE_RATING_FROM_RATING_SERVICE,
    containerFactory =
KafkaConstants.BASE_KAFKA_LISTENER_CONTAINER_FACTORY,
    properties =
{KafkaConstants.TARGET_DTO_FOR_DESERIALIZATION_PROPERTY})
```

Описание конфигурации получателя:

1 Аннотация `@KafkaListener` регистрирует метод как подписчика на определенный топик в рамках уникальной группы потребителей.

2 Параметр `properties` передает настройки десериализатору Jackson. Это позволяет автоматически преобразовывать входящий JSON-документ из топика брокера в строго типизированный Java-объект `AverageRatingResponseDto` без написания дополнительного шаблонного кода.

Логика обновления базы данных внутри метода-слушателя выглядит следующим образом:

```
Driver driver =
driverRepository.findByIdAndDeletedIsFalse(averageRatingResponse
Dto.refUserId())
        .orElseThrow(() -> new DriverNotFoundException(
            "Driver with id " +
averageRatingResponseDto.refUserId() + " not found"));
        driverMapper.partialUpdate(averageRatingResponseDto,
driver);
        driverRepository.save(driver);}
```

Описание кода обновления профиля:

1 Поиск профиля водителя в базе данных PostgreSQL по уникальному идентификатору `refUserId`, извлеченному из сообщения Kafka. В случае отсутствия активного профиля генерируется исключение `DriverNotFoundException`.

2 Обновление поля `averageRating` у найденной сущности водителя с фиксацией изменений в БД. Данный подход гарантирует согласованность данных в распределенной системе.

Последний шаг комплексного сценария заключается в регулярной (по расписанию) отправке водителям их персональной статистики. Процесс инициируется планировщиком задач в `driver-service` (класс `StatisticReportServiceImpl`).

Инициализация сбора данных происходит с помощью аннотации планировщика:

```
@Scheduled(cron = "0 0 0 1 * ?")
public void scheduledSenderStatisticToNotificationService(){
    var drivers = driverRepository.findAllByDeletedIsFalse();
```

Описание инициализации отчета:

1 Аннотация `@Scheduled` с Cron-выражением `0 0 0 1 * ?` обеспечивает автоматический запуск процесса ровно в полночь первого числа каждого месяца.

2 Извлечение списка всех активных водителей из базы данных для последующей агрегации статистики.

Далее для каждого пользователя происходит сбор данных из смежных сервисов и их отправка:

```
drivers.forEach(driver -> {  
    RideStatisticResponseDto rideStatistic =  
  
rideFeignClient.getRideStatisticForDriver(driver.getId());  
    GeneralUserStatisticDto generalStatistic =  
GeneralUserStatisticDto.builder()  
        .withRideStatistic(rideStatistic)  
        .withUser(driverMapper.toUserDto(driver))  
        .withUserType(AppConstants.USER_TYPE)  
        .build();  
    statisticSender.sendGeneralUserStatisticToNotificationService(generalStatistic);});}
```

Описание кода агрегации:

1 Для каждого водителя выполняется синхронный REST-запрос в rides-service через интерфейс rideFeignClient. Запрос возвращает историю поездок и подсчитанную среднюю стоимость за месяц.

2 Использование паттерна Builder для формирования комплексного объекта GeneralUserStatisticDto, объединяющего профиль водителя и его статистику поездок. Сформированный объект отправляется в топик Kafka для сервиса уведомлений.

В микросервисе notification-service сообщение перехватывается, после чего задача делегируется классу NotificationServiceImpl. Процесс подготовки электронного письма начинается с настройки параметров почтового отправления:

```
public void sendUserStatisticOnEmail (GeneralUserStatisticDto  
generalUserStatisticDto) {  
    try {  
        MimeMessage mimeMessage =  
mailSender.createMimeMessage();  
        MimeMessageHelper helper = new  
MimeMessageHelper(mimeMessage, true, "UTF-8");  
  
helper.setTo(generalUserStatisticDto.user().email());  
        helper.setSubject(AppConstants.MAIL_SUBJECT);  
  
helper.setText(buildHtmlGreetingText(generalUserStatisticDto).get(), true);
```

Описание настройки сообщения:

1 Создание объекта MimeMessage и использование обертки MimeMessageHelper для поддержки многокомпонентных сообщений (с вложениями) и кодировки UTF-8.

2 Установка адресата (извлеченного из DTO), темы письма и

формирование HTML-тела письма через вспомогательный метод `buildHtmlGreetingText()`.

Следующим этапом идет генерация PDF-отчета, его прикрепление к письму и отправка:

```
String                htmlReport                =
buildReportAsStringForPdf(generalUserStatisticDto);
                ByteArrayOutputStream                pdfStream                =
generatePdf(htmlReport);
                helper.addAttachment(AppConstants.REPORT_FILE_NAME,
                new
ByteArrayDataSource(pdfStream.toByteArray(),
AppConstants.PDF_ATTACHMENT_TYPE));
                mailSender.send(mimeMessage);
        } catch (Exception e) {
                log.error(e.getMessage(), e);
        }}
```

Описание кода генерации и отправки вложения:

1 Формирование разметки HTML-шаблона статистического отчета с подстановкой реальных данных водителя (список поездок, заработок, рейтинг).

2 Конвертация HTML-строки в байтовый поток PDF-документа путем вызова метода `generatePdf()`.

3 Прикрепление байтового массива в качестве вложения с указанием MIME-типа `application/pdf` и финальная отправка письма целевому пользователю через `JavaMailSender` по протоколу SMTP.

Сам механизм конвертации HTML в PDF изолирован во вспомогательном методе:

```
private                ByteArrayOutputStream                generatePdf(String
htmlContent) {
                ByteArrayOutputStream                outputStream                =                new
ByteArrayOutputStream();
                ITextRenderer renderer = new ITextRenderer();
                renderer.setDocumentFromString(htmlContent);
                renderer.layout();
                renderer.createPDF(outputStream);
                return outputStream;
        }
```

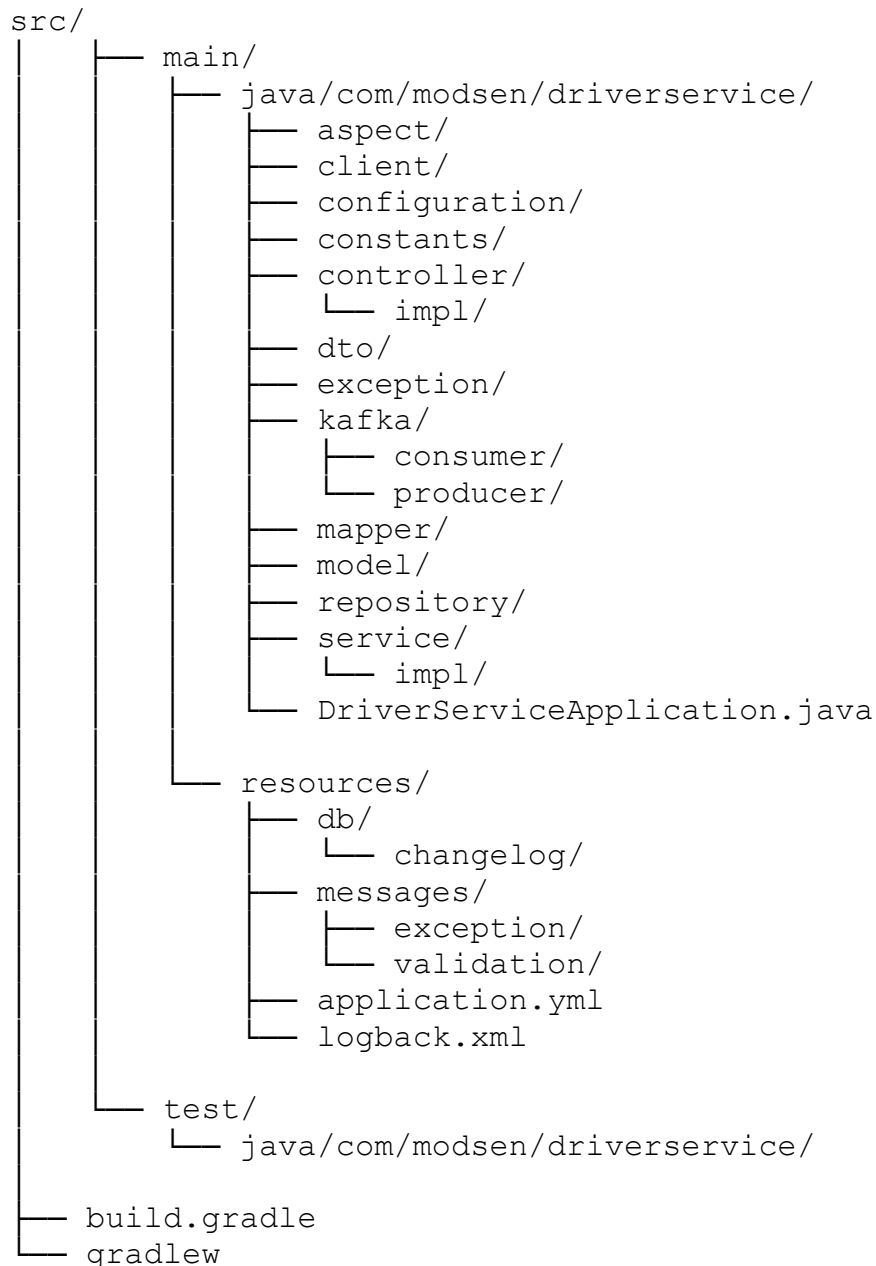
Описание кода PDF-преобразователя:

1 Инициализация выходного потока данных `ByteArrayOutputStream`.

2 Использование ядра рендеринга `ITextRenderer` (библиотека `Flying Saucer`). Метод `setDocumentFromString` загружает сформированный HTML, `layout()` просчитывает позиционирование элементов (шрифты, таблицы, отступы), а `createPDF()` записывает итоговый бинарный результат в выходной поток.

4.2 Структура базового микросервиса

Структура типового микросервиса приведена далее на примере сервиса водителей (driver-service):



Описание компонентов:

1 aspect/ содержит классы аспектно-ориентированного программирования (AOP), которые перехватывают вызовы методов для проверки прав доступа пользователя к запрашиваемым ресурсам (например, ValidateAccessToResourcesAspect).

2 client/ включает декларативные REST-клиенты для выполнения синхронных запросов к другим микросервисам, а также классы-обработчики ошибок для обеспечения отказоустойчивости.

3 configuration/ хранит классы конфигурации Spring-контекста,

определяющие настройки безопасности, работу с S3-хранилищем, кэшированием и документацией.

4 `constants/` содержит строковые константы и ключи файлов локализации для исключения дублирования текста в коде.

5 `controller/` реализует REST API приложения. В корневом пакете находятся Java-интерфейсы с аннотациями OpenAPI, а в подпакете `impl/` – их реализации.

6 `dto/` описывает структуры данных, используемые для передачи информации между клиентом и сервером. Здесь же применяются аннотации валидации (Jakarta Validation).

7 `exception/` определяет глобальный обработчик ошибок (@RestControllerAdvice) и пользовательские классы исключений, преобразующие внутренние ошибки в стандартизированные HTTP-ответы.

8 `kafka/` отвечает за асинхронное взаимодействие: конфигурацию брокера, слушателей топиков и отправителей сообщений.

9 `mapper/` содержит интерфейсы библиотеки MapStruct для автоматического двунаправленного преобразования объектов DTO в JPA-сущности.

10 `model/` содержит JPA-сущности, которые с помощью аннотаций Hibernate отображаются на соответствующие таблицы в реляционной базе данных.

11 `repository/` реализует логику доступа к данным, наследуя интерфейсы Spring Data JPA для генерации SQL-запросов к PostgreSQL.

12 `service/` содержит интерфейсы бизнес-логики приложения, а подпакет `impl/` – классы их реализации, взаимодействующие с репозиториями, клиентами и брокером сообщений.

13 `DriverServiceApplication.java` – основной класс, точка запуска приложения, инициализирующая Spring-контекст.

14 `resources/db/changelog/` содержит XML-скрипты миграций Liquibase для управления версионностью схемы базы данных.

15 `resources/messages/` хранит `properties`-файлы интернационализации для поддержки мультиязычных сообщений об ошибках и нарушениях валидации.

16 `resources/application.yml` описывает параметры подключения к базе данных, Kafka, серверу обнаружения (Eureka) и другим внешним зависимостям.

17 `resources/logback.xml` определяет правила форматирования и агрегации логов приложения для их отправки в систему мониторинга.

18 `test/` содержит интеграционные и модульные тесты, а также E2E сценарии, обеспечивающие проверку работоспособности каждого слоя приложения.

Такой подход к организации проекта обеспечивает четкое разделение уровней ответственности, предотвращает смешивание бизнес-логики с логикой маршрутизации и доступа к БД, а также значительно упрощает

сопровождение кода, масштабирование команды и тестирование отдельных модулей.

4.3 Описание REST API базовых микросервисов

Взаимодействие внешних клиентских приложений с серверной частью агрегатора такси осуществляется посредством архитектурного стиля REST.

Для автоматической генерации интерактивной документации и предоставления спецификации API разработчикам клиентской части используется библиотека `springdoc-openapi` (на базе стандарта OpenAPI 3.0 / Swagger). В рамках чистой архитектуры аннотации документирования и валидации JSR-380 вынесены в отдельные Java-интерфейсы (например, `DriverOperations`, `RideOperations`), что позволяет сохранить контроллеры (реализации) чистыми и сфокусированными исключительно на маршрутизации запросов.

Доступ к подавляющему большинству конечных точек (endpoints) защищен с помощью JWT-токенов. Ролевая модель доступа (RBAC) опирается на роли `ROLE_ADMIN`, `ROLE_DRIVER` и `ROLE_PASSENGER`. Дополнительно применяется кастомный аспект `@ValidateAccessToResources`, гарантирующий, что пользователи имеют доступ только к собственным данным.

4.3.1 API управления учетными записями

Данный API реализован в микросервисе `registration-service` и отвечает за интеграцию с сервером авторизации Keycloak и обеспечивает регистрацию новых участников системы, а также выдачу токенов доступа.

Базовый URL: `/api/v1/cab-aggregator`.

Список конечных точек:

1 POST `/signup` осуществляет регистрацию нового пользователя. Принимает DTO `SignUpDto` с персональными данными, паролем и ролью (`DRIVER` или `PASSENGER`). Иницирует создание профиля в смежных сервисах. Возвращает статус 201 `Created` при успешном создании или 409 `Conflict`, если пользователь с таким email/телефоном уже существует.

2 POST `/signin` выполняет аутентификацию пользователя. Принимает email и пароль. Возвращает объект `UserKeycloakTokenResponseDto`, содержащий `access_token`, `refresh_token` и время их жизни.

3 POST `/signin/admin` выполняет сервисную аутентификацию администратора. Используется для получения токенов межсервисного взаимодействия.

4.3.2 API управления профилями водителей и автопарком

API микросервиса водителей разделено на три логических контроллера: управление профилем водителя, управление автомобилями и работа с медиаконтентом (аватарами).

Базовый URL для водителей: /api/v1/drivers.

Базовый URL для автомобилей: /api/v1/cars.

Список конечных точек водителя:

1 POST /api/v1/drivers выполняет создание профиля (вызывается внутренним запросом от registration-service).

2 GET /api/v1/drivers возвращает список всех водителей. Поддерживает пагинацию через Query-параметры.

3 GET /api/v1/drivers/{driverId} возвращает данные конкретного водителя по его идентификатору.

4 GET /api/v1/drivers/{driverId}/cars возвращает данные водителя по его идентификатору вместе со списком его машин.

5 PUT /api/v1/drivers/{driverId} выполняет полное обновление данных профиля. Доступно самому водителю или администратору.

6 DELETE /api/v1/drivers/{driverId} логическое удаление профиля водителя (установление флага deleted = true).

Список конечных точек автомобилей:

1 POST /api/v1/cars/drivers/{driverId} выполняет добавление нового автомобиля водителю. Доступно только администратору (ROLE ADMIN).

2 PUT /api/v1/cars/{carId}/drivers/{driverId} выполняет обновление технических характеристик машины.

3 GET /api/v1/cars/{carId} получает данные автомобиля по идентификатору.

4 DELETE /api/v1/cars/{carId} убирает привязку автомобиля к водителю.

Список конечных точек медиаконтента:

1 POST /api/v1/drivers/{id}/avatars/upload выполняет загрузку изображения пользователя. Файл валидируется на расширение (.png, .jpeg) и размер, после чего отправляется в MinIO.

2 GET /api/v1/drivers/{id}/avatars возвращает поток байт изображения для отображения на клиентской части приложения.

3 DELETE /api/v1/drivers/{id}/avatars удаляет изображение профиля.

4.3.3 API управления профилями пассажиров

Функционал данного API аналогичен сервису водителей, но адаптирован под бизнес-сущность пассажира и не содержит логики работы с автомобилями.

Базовый URL: /api/v1/passengers.

Включает в себя стандартный набор CRUD-операций:

1 POST /api/v1/passengers осуществляет создание профиля.

2 GET /api/v1/passengers возвращает список пассажиров (с пагинацией).

3 GET /api/v1/passengers/{passengerId} возвращает информацию о пассажире по его идентификатору.

4 PUT /api/v1/passengers/{passengerId} обновляет данные о пассажире.

5 DELETE /api/v1/passengers/{passengerId} удаляет профиль пассажира.

Также присутствуют эндпоинты для загрузки, получения и удаления аватаров по пути /api/v1/passengers/{id}/avatars.

4.3.4 API управления поездками

Данный API является наиболее высоконагруженным, так как обеспечивает работу приложения в режиме реального времени.

Базовый URL: /api/v1/rides.

Список конечных точек:

1 POST /api/v1/rides создает заявку на поездку. В теле запроса передаются driverId, passengerId, departureAddress и destinationAddress. Метод рассчитывает начальную стоимость поездки, фиксирует время создания и устанавливает статус CREATED.

2 PATCH /api/v1/rides/{rideId}/status выполняет изменение статуса существующей поездки. Данный метод использует метод PATCH, так как обновляет только одно поле ресурса. Статус проверяется конечным автоматом. Доступно водителю, пассажиру (для отмены) и администратору.

3 PUT /api/v1/rides/{rideId} выполняет обновление параметров поездки (например, изменение адреса назначения по ходу поездки).

4 GET /api/v1/rides/{rideId} возвращает детальную информацию о конкретной поездке.

5 GET /api/v1/rides возвращает все поездки в системе (с пагинацией).

6 GET /api/v1/rides/drivers/{driverId} / GET /api/v1/rides/passengers/{passengerId} возвращает историю поездок для конкретного участника системы.

7 GET /api/v1/rides/drivers/{driverId}/statistics возвращает агрегированную статистику по поездкам водителя.

4.3.5 API системы рейтингов и отзывов

Для логического разделения предусмотрено два набора эндпоинтов: для оценки водителей и для оценки пассажиров.

Базовые URL: /api/v1/drivers-ratings и /api/v1/passengers-ratings.

Оба контроллера предоставляют идентичный набор интерфейсов:

1 POST / выполняет публикацию нового отзыва. В теле передаются rideId, оценка от 1 до 5 и текстовый комментарий. Перед сохранением система проверяет факт завершения поездки через вызов rides-service.

2 PUT /{id} осуществляет редактирование комментария или оценки.

3 GET /{id} возвращает конкретный отзыв по его идентификатору.

4 DELETE /{id} выполняет удаление отзыва модератором.

5 GET /users/{refUserId} возвращает список отзывов пользователя.

5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

В данном разделе рассмотрено тестирование разработанной распределенной системы, представляющей собой базовую микросервисную инфраструктуру для приложения агрегатора такси. Проведение испытаний является важным этапом разработки, позволяющим выявить программные дефекты до ввода системы в эксплуатацию, проверить соответствие реализованного функционала заявленным требованиям, а также оценить отказоустойчивость, безопасность и корректность межсервисного взаимодействия.

Поскольку разрабатываемый программный продукт является инфраструктурным решением, тестирование охватывает автоматизированные проверки (модульные, интеграционные, сквозные), ручное тестирование REST API через единый шлюз, а также проверку работоспособности вспомогательных UI-интерфейсов инфраструктурных узлов.

5.1 Конфигурация тестовой среды

Для проведения испытаний разработанного программного средства была подготовлена локальная тестовая среда, включающая программные и аппаратные компоненты, обеспечивающие корректную работу всех контейнеризированных модулей системы. Ввиду одновременного запуска множества ресурсоемких инфраструктурных компонентов, к аппаратному обеспечению тестового стенда предъявлялись строгие требования по объему оперативной памяти и вычислительной мощности. Конфигурация этой среды представлена в таблицах 5.1 и 5.2.

Таблица 5.1 – Аппаратная конфигурация тестовой среды

Компонент	Описание
Процессор	Intel Core Ultra 7 155U Processor (12 Cores)
Оперативная память	32 ГБ DDR5
Накопитель	SSD NVMe 512 ГБ
Сетевое подключение	Локальная сеть (localhost), доступ в интернет

Таблица 5.2 – Программная конфигурация тестовой среды

Компонент	Версия
Операционная система	Windows 10 Профессиональная
Среда выполнения Java	JDK 21 (Eclipse Temurin)
Платформа контейнеризации	Docker Desktop 4.40.0 (Docker Engine 28.0.4)
Инструмент оркестрации	Docker Compose v2.34.0
База данных	PostgreSQL 17
Брокер сообщений	Apache Kafka 2.8.1
Сервер авторизации	Keycloak 26.0.7
In-memory хранилище	Redis 7.4.2

5.2 Автоматизированное тестирование базовых микросервисов

Для гарантии надежности бизнес-логики и предотвращения регрессионных ошибок при дальнейшем масштабировании инфраструктуры, в проекте реализована многоуровневая пирамида тестирования. Для наглядной демонстрации процесса автоматизированного тестирования был выбран центральный модуль системы – микросервис управления поездками.

5.2.1 Модульные и интеграционные тесты

На нижнем уровне пирамиды реализованы модульные тесты, проверяющие изолированную работу отдельных слоев (сервисов, контроллеров). Тестирование выполнено с использованием фреймворка JUnit 5 и библиотеки для заглушек Mockito.

В ходе испытаний сервисного слоя все внешние зависимости, такие как репозитории баз данных и Feign-клиенты, были заменены заглушками (@Mock). Это позволило проверить исключительные ситуации (например, выброс `RideNotFoundException` или `InvalidInputStatusException`) без обращения к реальной БД. Слой контроллеров протестирован с использованием аннотации `@WebMvcTest` и утилиты `MockMvc`.

Интеграционное тестирование направлено на проверку корректности взаимодействия микросервиса с реальными инфраструктурными компонентами. В проекте для этого применена библиотека `Testcontainers`.

В рамках испытаний динамически поднимались легковесные изолированные Docker-контейнеры с PostgreSQL, Redis и Keycloak. Для проверки межсервисного взаимодействия по REST API использовался инструмент `WireMock` (@AutoConfigureWireMock), который имитировал ответы сторонних сервисов.

Результаты прохождения модульных и интеграционных тестов представлены на рисунке 5.1.

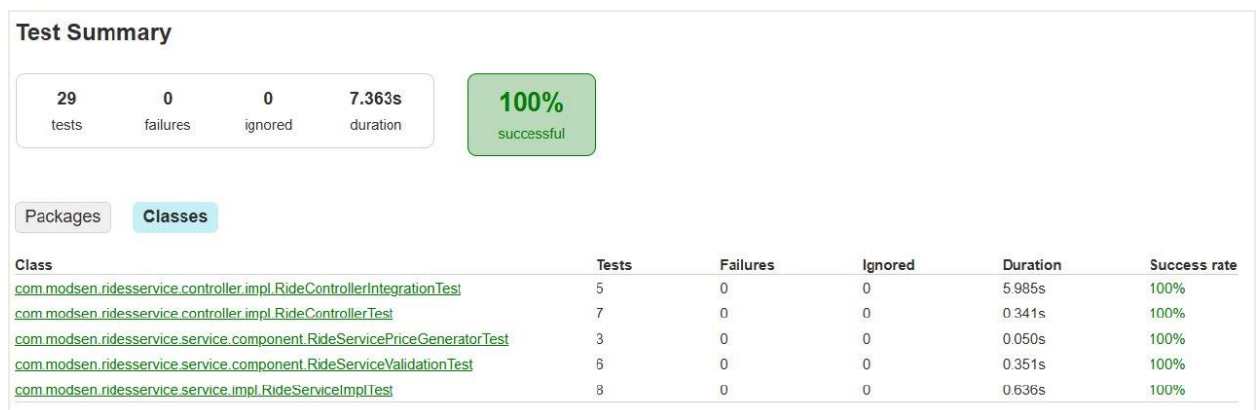


Рисунок 5.1 – Результаты выполнения модульных и интеграционных тестов

5.2.2 Сквозное тестирование (E2E)

На вершине пирамиды реализовано E2E тестирование с использованием

методологии BDD (Behavior-Driven Development) на базе фреймворка Cucumber. Сценарии использования написаны на понятном человеческом языке (синтаксис Gherkin) в .feature файлах. Выполнение реальных HTTP-запросов к работающим контейнерам (развернутым через docker-compose-e2e.yml) производилось библиотекой REST Assured. Результаты прохождения сквозных BDD-сценариев представлены на рисунке 5.2.

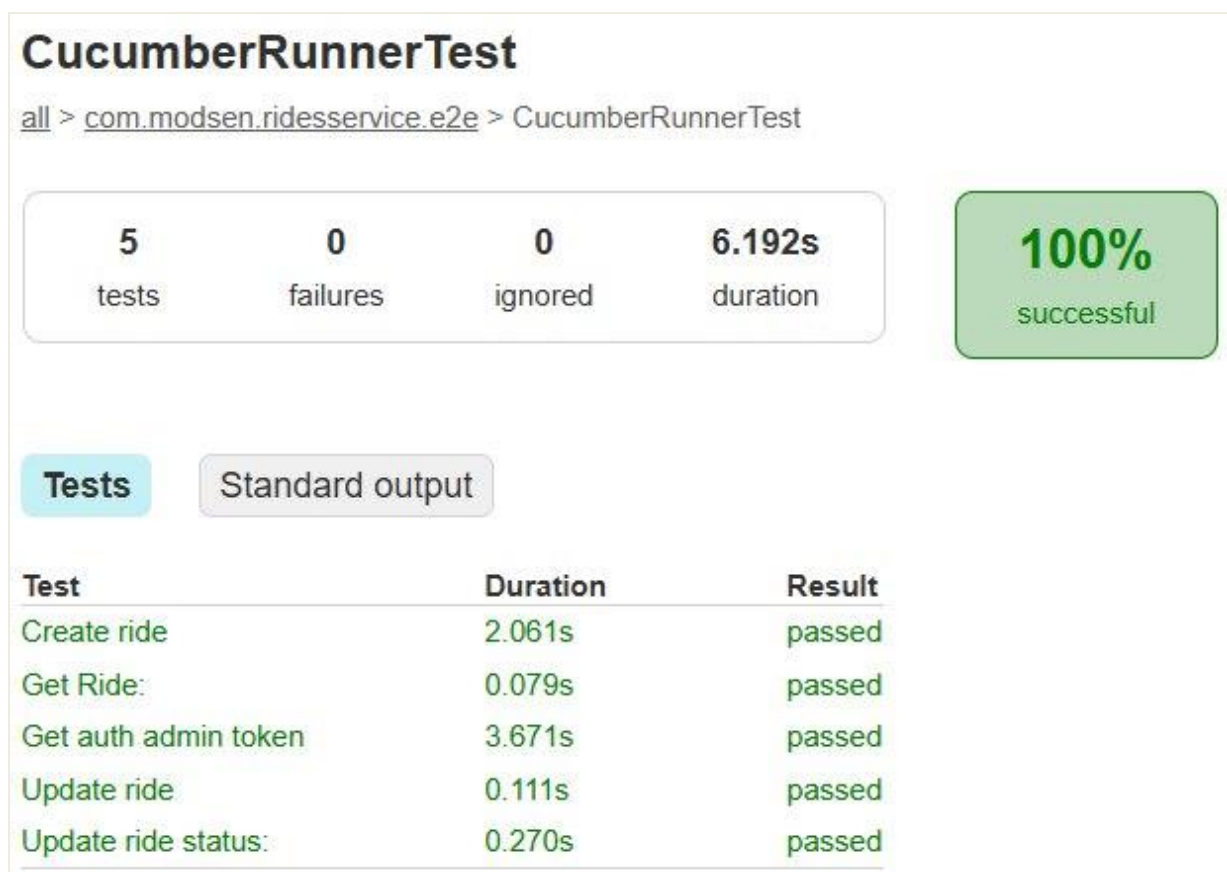


Рисунок 5.2 – Результаты выполнения сквозных E2E-тестов

5.3 Ручное тестирование REST API

Для проверки доступности системы извне и оценки удобства использования API со стороны клиентских разработчиков, было проведено ручное тестирование с использованием встроенного инструмента Swagger UI. Данный инструмент автоматически генерирует интерактивную спецификацию на основе исходного кода (стандарт OpenAPI 3.0), позволяя отправлять реальные HTTP-запросы, передавать JWT-токены авторизации и просматривать структуру возвращаемых данных прямо из браузера. Благодаря настройкам API Gateway, документация всех микросервисов агрегирована в едином веб-интерфейсе, доступном по единому адресу.

Все запросы маршрутизируются исключительно через шлюз (API Gateway), который прозрачно перенаправляет трафик на нужный микросервис, предварительно выполняя фильтрацию и применяя глобальные

CORS-политики. Описание ключевых маршрутов системы приведено в таблице 5.3.

Таблица 5.3 – Описание маршрутов микросервисной инфраструктуры

№	Сервис	Маршрут	Задачи
1	registration-service	/api/v1/cab-aggregator/signup	Регистрация пользователя
2		/api/v1/cab-aggregator/signin	Вход в аккаунт
3	driver-service	/api/v1/drivers/{id}	Управление водителем
4		/api/v1/drivers/{id}/avatars	Управление фото водителя
5	passenger-service	/api/v1/passengers/{id}	Управление пассажиром
6	rides-service	/api/v1/rides	Управление поездками
7		/api/v1/rides/{id}/status	Изменение статуса поездки
8	rating-service	/api/v1/drivers-ratings	Создание отзыва о водителе

Для большинства маршрутов были проведены ручные тесты, симулирующие глобальный бизнес-сценарий работы агрегатора (от регистрации до выставления оценки). Результаты приведены в таблице 5.4.

Таблица 5.4 – Тестирование маршрутов REST API

№	Сценарий	Ожидаемый результат	Полученный результат
1	2	3	4
1	Регистрация пользователя	Ответ с кодом 201 Created. Пользователь успешно сохранен в Keycloak и БД пассажиров	Соответствует ожидаемому результату
1	Регистрация дубликата	Ответ с кодом 409 Conflict и сообщением об ошибке от сервера авторизации	Соответствует ожидаемому результату
2	Авторизация	Ответ с кодом 200 OK, содержащий access_token и refresh_token	Соответствует ожидаемому результату
4	Загрузка аватара (неверный формат)	Ответ с кодом 415 Unsupported Media Type и сообщением «Unsupported file»	Соответствует ожидаемому результату

Продолжение таблицы 5.4

1	2	3	4
6	Доступ без токена	Ответ с кодом 401 Unauthorized	Соответствует ожидаемому результату
6	Создание поездки	Ответ с кодом 201 Created. В теле ответа JSON объект поездки со статусом CREATED и сгенерированной стоимостью	Соответствует ожидаемому результату
7	Изменение статуса поездки (пропуск этапов)	Ответ с кодом 409 Conflict. Сообщение об ошибке: «Cannot update status from CREATED to COMPLETED»	Соответствует ожидаемому результату
8	Оценка поездки	Ответ с кодом 201 Created. Иницируется асинхронная отправка сообщения в Kafka для пересчета рейтинга	Соответствует ожидаемому результату

Успешное выполнение описанных сценариев подтверждает надежность защиты эндпоинтов, корректность HTTP-маршрутизации через API Gateway и высокую информативность возвращаемых ошибок.

5.4 Тестирование инфраструктурных компонентов

Инфраструктура микросервисного приложения включает в себя ряд готовых Enterprise-решений, предоставляющих собственные панели управления веб-интерфейсом. В ходе испытаний была проверена их доступность и корректность интеграции с микросервисами.

5.4.1 Тестирование сервера авторизации (Keycloak)

Для проверки управления пользователями был осуществлен вход в панель администратора Keycloak, развернутого в Docker-контейнере (рисунок 5.3).

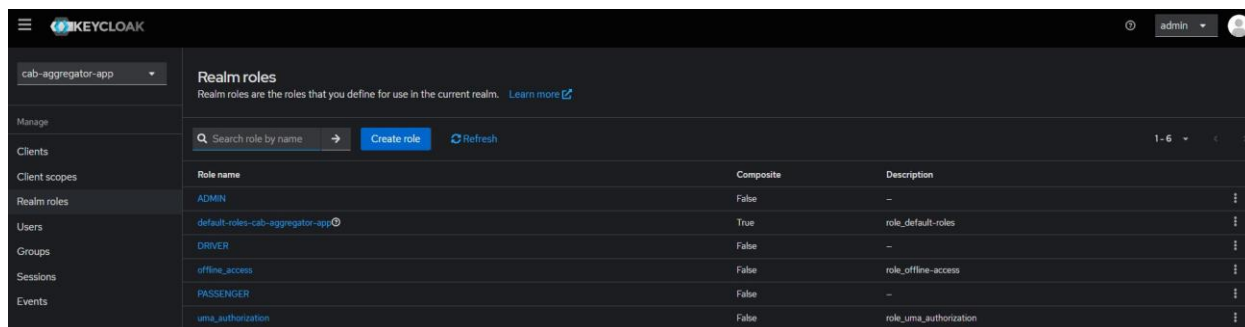


Рисунок 5.3 – Панель управления ролями пользователей в Keycloak Admin Console

В ходе проверки подтверждено:

- успешный импорт настроек Realm (cab-aggregator-app-realm.json) при старте контейнера;
- наличие автоматически созданных ролей (DRIVER, PASSENGER, ADMIN);
- корректное появление новых пользователей в интерфейсе Keycloak после выполнения запроса регистрации через REST API.

5.4.2 Тестирование системы мониторинга (Grafana, Loki, Tempo)

Важнейшим аспектом распределенной системы является наблюдаемость (Observability). Для тестирования этого функционала был сгенерирован тестовый трафик на API, после чего производился анализ данных в интерфейсе Grafana (рисунок 5.4).

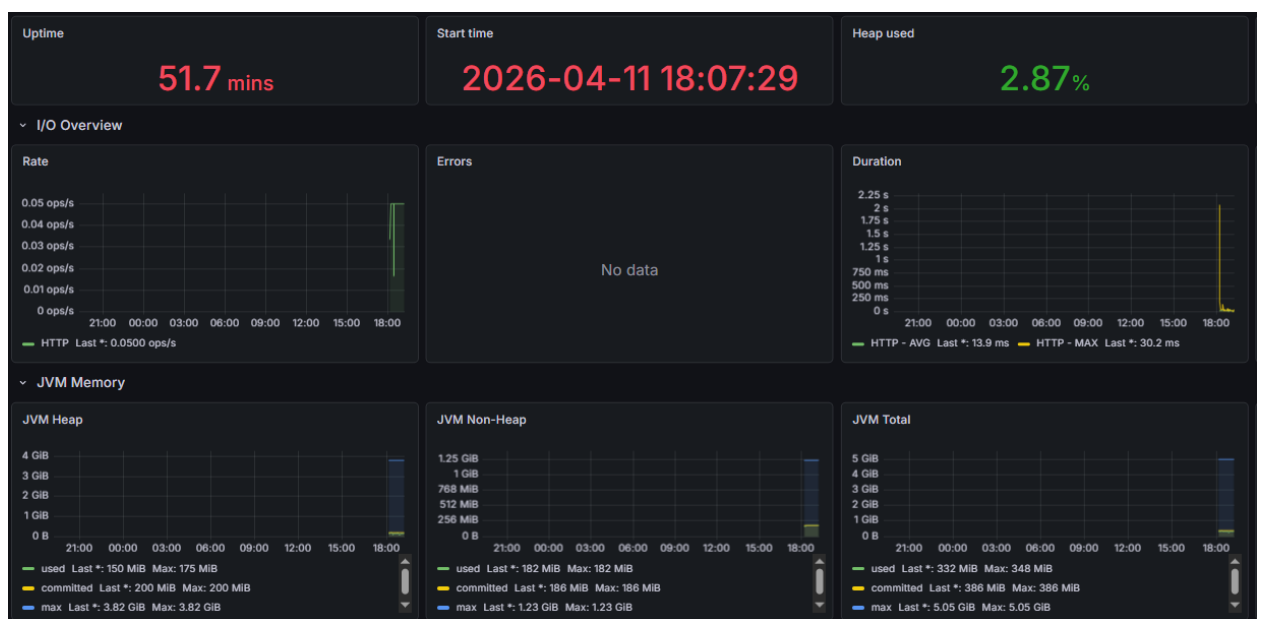


Рисунок 5.4 – Панель с метриками микросервисов в Grafana

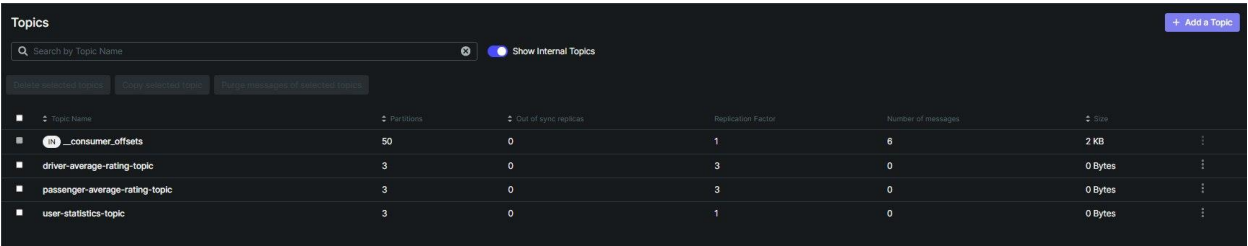
Тестирование подтвердило:

- сбор метрик: графики нагрузки на процессор (CPU), потребления оперативной памяти (JVM Heap) и количества HTTP-запросов корректно отображаются благодаря интеграции VictoriaMetrics;
- централизованное логирование: логи со всех микросервисов агрегируются через Loki.
- распределенная трассировка: система Tempo успешно фиксирует время прохождения запроса через узлы сети, позволяя визуально выявлять задержки в межсервисном взаимодействии.

5.4.3 Тестирование брокера сообщений (Kafka UI)

Для проверки асинхронного взаимодействия был использован веб-интерфейс Kafka-UI. Выполнялся сценарий отправки отзыва пассажиром, что инициировало пересчет рейтинга водителя.

На рисунке 5.5 отображены созданные топики Kafka в инфраструктуре агрегатора такси.



The screenshot shows the 'Topics' page in the Kafka UI. It includes a search bar, a 'Show Internal Topics' toggle, and a table of topics. The table has columns for Topic Name, Partitions, Out of sync replicas, Replication Factor, Number of messages, and Size. The topics listed are _consumer_offsets, driver-average-rating-topic, passenger-average-rating-topic, and user-statistics-topic.

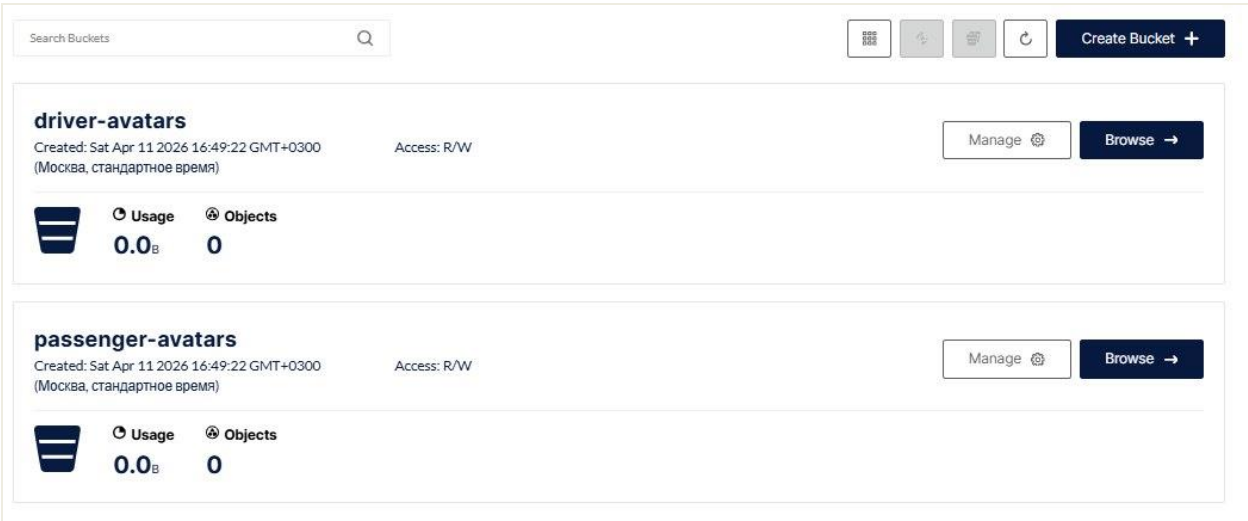
Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size
_consumer_offsets	50	0	1	6	2 KB
driver-average-rating-topic	3	0	3	0	0 Bytes
passenger-average-rating-topic	3	0	3	0	0 Bytes
user-statistics-topic	3	0	1	0	0 Bytes

Рисунок 5.5 – Созданные топики в Kafka UI

- В ходе проверки подтверждено:
- топики (например, driver-average-rating-topic и user-statistics-topic) создаются автоматически с заданным количеством партиций и реплик;
 - сообщения успешно сериализуются в формат JSON и поступают в топик;
 - группа потребителей микросервисов корректно считывают сообщения (смещение фиксируется, задержка равна нулю).

5.4.4 Тестирование S3-совместимого хранилища (MinIO)

Для хранения пользовательского медиаконтента (аватаров водителей и пассажиров) в распределенной инфраструктуре предусмотрено использование объектного хранилища MinIO. Тестирование данного компонента проводилось как через REST API микросервисов, так и с помощью встроенной панели управления хранилищем MinIO Console (рисунок 5.6).



The screenshot shows the MinIO Console interface. It features a search bar, a 'Create Bucket' button, and two bucket entries: 'driver-avatars' and 'passenger-avatars'. Each entry shows creation time, access permissions, and usage statistics (Usage and Objects).

Bucket Name	Created	Access	Usage	Objects
driver-avatars	Sat Apr 11 2026 16:49:22 GMT+0300 (Москва, стандартное время)	R/W	0.0 B	0
passenger-avatars	Sat Apr 11 2026 16:49:22 GMT+0300 (Москва, стандартное время)	R/W	0.0 B	0

Рисунок 5.6 – Интерфейс панели управления объектным хранилищем MinIO

- В ходе проверки работы объектного хранилища было подтверждено:
- инициализация бакетов: при запуске Docker-контейнера

автоматически создаются логические хранилища (бакеты) `driver-avatars` и `passenger-avatars`, заданные через переменные окружения (`MINIO_DEFAULT_BUCKETS`);

- загрузка и хранение объектов: при успешном выполнении POST-запроса на загрузку аватара через микросервисы профилей, файл корректно сохраняется в соответствующий бакет и в интерфейсе MinIO отображаются правильные метаданные объекта (MIME-тип `image/jpeg` или `image/png`, размер файла и дата загрузки);

- доступность данных: при GET-запросе к микросервису файл успешно извлекается из бакета MinIO и передается клиенту в виде потока байтов (`InputStreamResource`) с заголовком `Content-Disposition: inline`, что позволяет корректно отображать изображение в браузере или мобильном приложении;

- удаление объектов: при отправке DELETE-запроса на удаление аватара, файл гарантированно удаляется из бакета, высвобождая дисковое пространство, что исключает появление «мусорных» файлов в файловой системе.

Успешные результаты тестирования подтверждают, что интеграция микросервисов с MinIO выполнена корректно, а само хранилище способно надежно и быстро обрабатывать медиафайлы в условиях распределенной архитектуры агрегатора такси.

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ ИНФРАСТРУКТУРЫ МИКРОСЕРВИСНОГО ПРИЛОЖЕНИЯ ДЛЯ АГРЕГАТОРА ТАКСИ

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Дипломный проект представляет собой программную систему, предназначенную для реализации базовой инфраструктуры микросервисного приложения агрегатора такси.

Цель разработки данного проекта – создание масштабируемой и технологически устойчивой программной основы, обеспечивающей возможность построения современных цифровых сервисов для организации пассажирских перевозок и взаимодействия между пользователями платформы.

Разработка подобных программных продуктов осуществляется ИТ-компаниями, использующими продуктовую модель бизнеса, при которой разработчик является владельцем программного решения и осуществляет его дальнейшее развитие, поддержку и распространение на рынке цифровых сервисов.

Целевую аудиторию программного продукта составляют организации, занимающиеся разработкой цифровых сервисов в сфере транспортных услуг, а также компании, заинтересованные в создании собственных платформ для организации поездок. Дополнительно потенциальными пользователями системы могут выступать стартап-команды и технологические компании, разрабатывающие сервисы мобильности и платформы для управления транспортными потоками.

В последние годы наблюдается значительный рост популярности онлайн-сервисов заказа поездок. Подобные платформы позволяют автоматизировать процесс взаимодействия между пассажирами и водителями, обеспечивают эффективное распределение заказов и сокращают время ожидания транспорта. В условиях высокой конкуренции на рынке транспортных услуг компании стремятся использовать современные технологические решения, обеспечивающие высокую производительность, масштабируемость и отказоустойчивость программных систем.

В рамках данного дипломного проекта разрабатывается программное средство, представляющее собой базовую микросервисную инфраструктуру агрегатора такси. Основное назначение системы заключается в предоставлении инфраструктурной основы для создания распределенного программного приложения, способного обрабатывать большое количество пользовательских запросов и обеспечивать устойчивую работу сервисов.

Использование микросервисной архитектуры позволяет разделить систему на независимые компоненты, каждый из которых выполняет строго определенные функции. Такой подход значительно повышает гибкость

разработки, упрощает масштабирование системы и обеспечивает более высокую устойчивость программного решения к отказам отдельных компонентов.

7.2 Расчет инвестиций в разработку программного средства

7.2.1 Расчет затрат на основную заработную плату разработчиков.

Для определения объемов инвестиций в разработку программного средства, ориентированного на использование в цифровых сервисах транспортной сферы, прежде всего необходимо определить состав команды разработчиков и рассчитать затраты на оплату их труда.

Состав исполнителей был сформирован исходя из технологической сложности разрабатываемой системы, которая предполагает проектирование микросервисной архитектуры, разработку серверной логики приложения, а также организацию инфраструктуры развертывания и сопровождения сервисов.

В разработке программного средства задействованы три специалиста:

- архитектор микросервисной архитектуры;
- старший инженер-программист;
- инженер по развертыванию инфраструктуры (DevOps).

Архитектор микросервисной архитектуры выполняет роль старшего инженера-программиста и системного архитектора проекта. В его основные обязанности входит проектирование архитектуры микросервисного приложения, разработка ключевых сервисов системы, построение взаимодействия между компонентами инфраструктуры, а также обеспечение масштабируемости и устойчивости программной платформы.

Старший инженер-программист отвечает за реализацию серверной логики отдельных микросервисов системы, разработку API для взаимодействия между компонентами приложения, работу с базами данных, а также реализацию бизнес-логики системы обработки заказов.

Инженер по развертыванию инфраструктуры (DevOps) обеспечивает создание и настройку инфраструктуры развертывания приложения. В его задачи входит организация контейнеризации сервисов, настройка систем оркестрации, конфигурация процессов непрерывной интеграции и доставки программного обеспечения, а также обеспечение мониторинга и стабильности работы системы.

Затраты на основную заработную плату рассчитаны по формуле

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (7.1)$$

где $K_{\text{пр}}$ – коэффициент премий и иных стимулирующих выплат;

n – категории исполнителей, занятых разработкой программного средства;

$Z_{\text{чи}}$ – часовая заработная плата исполнителя i -го исполнителя (руб);

t_i – трудоемкость работ, выполняемых исполнителем i -м исполнителя, (ч).

Для расчета заработной платы были использованы данные о среднем уровне оплаты труда специалистов в сфере информационных технологий на территории Республики Беларусь.

Согласно анализу рынка труда, медианная заработная плата инженеров составляет около 1500 долларов США, а специалистов DevOps – около 1600 долларов США [22].

По состоянию на 8 марта 2026 года официальный курс доллара США, установленный Национальным банком Республики Беларусь, составляет 2,9013 белорусских рубля [23].

Исходя из этого месячный оклад backend разработчика составляет 4351,95 руб., а DevOps-инженера — 4642,76 руб.

Расчет часового оклада производится путем деления месячного оклада на количество рабочих часов в месяце. Согласно нормативам Министерства труда Республики Беларусь, норма рабочего времени составляет 176 часов [24]. Затраты на основную заработную плату сотрудников представлены в таблице 7.1

Таблица 7.1 – Затраты на основную заработную плату

Наименование должности разработчика	Месячный оклад, р	Часовой оклад, р	Трудоемкость работ, ч	Итого, р
Старший инженер-программист	4 351,95	24,72	160	3 956,32
Старший инженер-программист	4 351,95	24,72	160	3 956,32
DevOps-инженер	4 642,76	26,38	80	2 110,35
Итого				10 022,99
Премия и иные стимулирующие выплаты (0%)				10%
Всего затраты на основную заработную плату разработчиков				11 025,29

7.2.2 Расчет затрат на дополнительную заработную плату по формуле

$$З_д = \frac{З_о \cdot Н_д}{100}, \quad (7.2)$$

где $Н_д$ – норматив дополнительной заработной платы.

Значение норматива дополнительной заработной платы принимает 10 %.

7.2.3 Расчет отчислений на социальные нужды. Величина обязательных

отчислений устанавливалась в соответствии с действующей на момент проведения расчетов ставкой социальных взносов. По состоянию на март 2026 года совокупный норматив отчислений составлял 35%, включая 29%, направляемых на пенсионное обеспечение, и 6%, предусмотренных для целей социального страхования. Для определения размера отчислений на социальные нужды применяется формула

$$P_{\text{соц}} = \frac{(Z_o + Z_d) \cdot H_{\text{соц}}}{100}, \quad (7.3)$$

где $H_{\text{соц}}$ – норматив отчислений в ФСЗН.

7.2.4 Расчет затрат на прочие расходы определяется при помощи норматива прочих расчетов. Эта величина имеет значение 30%. Для определения суммы прочих расходов применяется формула

$$P_{\text{пр}} = \frac{Z_o \cdot H_{\text{пр}}}{100}, \quad (7.4)$$

где $H_{\text{пр}}$ – норматив прочих расходов.

7.2.5 Расчет расходов на реализацию. Для того, чтобы рассчитать расходы на реализацию инфраструктуры микросервисного приложения, необходимо знать норматив расходов на нее. Принимаем значение норматива равным 4%. Для определения размера расходов на реализацию применяется формула

$$P_p = \frac{Z_o \cdot H_{\text{пр}}}{100}. \quad (7.5)$$

7.2.6 Расчет общей суммы инвестиций на разработку. Общая сумма инвестиций формируется путем суммирования следующих статей: основная заработная плата разработчиков, дополнительная заработная плата разработчиков, отчисления на социальные нужды, расходы на реализацию, а также прочие расходы. Детальный анализ каждой статьи позволяет получить точные сведения о затратах на сотрудников. Общая величина определяется по формуле

$$Z_p = Z_o + Z_d + P_{\text{соц}} + P_{\text{пр}} + P_p. \quad (7.6)$$

В результате, величина затрат на разработку инфраструктуры микросервисного приложения для агрегатора такси высчитывается по указанной выше формуле и указана в таблице 7.2. Стоит учесть, что все расчеты производятся в той же таблице.

Таблица 7.2 – Затраты на разработку

Название статьи затрат	Формула/таблица для расчета	Значение, р.
1 Основная заработная плата разработчиков	Таблица 7.1	11 025,29
2 Дополнительная заработная плата разработчиков	$З_d = \frac{11\,025,29 \cdot 10}{100}$	1 102,53
3 Отчисление на социальные нужды	$P_{\text{соц}} = \frac{(11\,025,29 + 1\,102,53) \cdot 35}{100}$	4 244,74
4 Прочие расходы	$P_{\text{пр}} = \frac{11\,025,29 \cdot 30}{100}$	3 315,69
5 Расходы на реализацию	$P_p = \frac{11\,025,29 \cdot 4}{100}$	441,01
6 Общая сумма затрат на разработку и реализацию	$З_p = 11\,025,29 + 1\,102,53 + 4\,244,74 + 3\,315,69 + 441,01$	20 129,26

7.3 Расчет экономического эффекта от реализации программного средства на рынке

Экономический эффект организации-разработчика при использовании продуктовой бизнес-модели представляет собой прирост чистой прибыли, величина которой напрямую зависит от прогнозируемого объема продаж, цены реализации и рентабельности продукта. В отличие от программных решений, создаваемых по индивидуальному заказу, данное программное средство ориентировано на массовый рынок разработчиков и компаний, заинтересованных в создании собственных сервисов агрегаторов такси, и может монетизироваться по модели Freemium.

В рамках данной модели пользователям предоставляется базовый бесплатный доступ к инфраструктуре микросервисной платформы, позволяющий осуществлять разработку и тестирование собственных сервисов. Расширенные возможности системы, такие как увеличение доступных вычислительных ресурсов, расширенные инструменты мониторинга, масштабирование инфраструктуры и техническая поддержка, могут предоставляться в рамках платной подписки.

Для обоснования экономического результата принят прогнозируемый годовой объем продаж в размере 40 платных подписок при отпускной цене 6000,00 белорусских рублей за годовой доступ. С учетом распространенности цифровых сервисов заказа поездок и роста интереса компаний к созданию собственных платформ городской мобильности, плановый показатель в 40 подписок охватывает лишь небольшую часть потенциального рынка, что

подтверждает реализуемость проекта и наличие возможностей для дальнейшего масштабирования.

Рентабельность продаж программного продукта принимается на уровне 20 процентов в соответствии с рекомендациями по оценке эффективности программных проектов.

При вычислении величины прироста чистой прибыли обязательным этапом является корректировка на сумму налога на добавленную стоимость. Механизм расчета данного налогового вычета базируется на следующей формуле

$$\text{НДС} = \frac{\text{Ц}_{\text{отп}} \cdot N \cdot \text{Н}_{\text{д.с}}}{100\% + \text{Н}_{\text{д.с}}}, \quad (7.7)$$

где N – количество оплат за подписку за год, шт.;

$\text{Ц}_{\text{отп}}$ – отпускная цена разовой подписки, р.;

$\text{Н}_{\text{д.с}}$ – ставка налога на добавленную стоимость, %.

Ставка налога на добавленную стоимость по состоянию на 8 марта 2026 года, в соответствии с действующим законодательством Республики Беларусь, составляет 20% [25]. Используя данное значение, посчитаем НДС:

$$\text{НДС} = \frac{6000,00 \cdot 40 \cdot 20\%}{100\% + 20\%} = 40\,000,00 \text{ р.}$$

Посчитав налог на добавленную стоимость, можно рассчитать прирост чистой прибыли, которую получит разработчик от продажи программного продукта. Для этого используется формула:

$$\Delta \Pi_{\text{ч}}^{\text{р}} = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot \text{Р}_{\text{пр}} \cdot \left(1 - \frac{\text{Н}_{\text{п}}}{100}\right), \quad (7.8)$$

где N – количество оплат за подписку за год, шт.;

$\text{Ц}_{\text{отп}}$ – отпускная цена разовой подписки, р.;

НДС – сумма налога на добавленную стоимость, р.;

$\text{Н}_{\text{п}}$ – ставка налога на прибыль, %;

$\text{Р}_{\text{пр}}$ – рентабельность продаж подписок.

Ставка налога на прибыль, согласно действующему законодательству, равна 20%. Эти показатели являются ключевыми для оценки финансовой эффективности проекта. Они напрямую влияют на доходность инвестиций. Рентабельность продаж копий взята в размере 20%. Зная ставку налога и рентабельность продаж подписок, рассчитывается прирост чистой прибыли для разработчика:

$$\Delta \Pi_{\text{ч}}^{\text{р}} = (6000,00 \cdot 40 - 40\,000,00) \cdot 20\% \cdot \left(1 - \frac{20}{100}\right) = 32\,000,00 \text{ р.}$$

7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Для оценки экономической эффективности разработки и реализации программного средства на рынке необходимо сравнить затраты на разработку программного продукта с ожидаемым годовым экономическим эффектом, выраженным в приросте чистой прибыли.

эффекта, поэтому можно сделать вывод, что такие инвестиции окупятся менее, чем за один год.

Исходя из этого, оценка экономической эффективности инвестиций производится при помощи расчета рентабельности инвестиций. Формула для расчета ROI:

$$ROI = \frac{\Delta\P_{\text{ч}}^p - \text{З}_p}{\text{З}_p} \cdot 100\%, \quad (7.9)$$

где $\Delta\P_{\text{ч}}^p$ – прирост чистой прибыли, полученной от реализации программного средства на рынке информационных технологий, р.;

З_p – затраты на разработку и реализацию программного средства, р.

$$ROI = \frac{32\,000,00 - 20\,129,26}{20\,129,26} \cdot 100\% = 59,0\%. \quad (7.10)$$

По итогам расчета, рентабельность инвестиций в разработку является высокой. Она в 3,85 раз превосходит ставку по долгосрочным депозитам, которая составляет 15,31 %, в соответствии с информацией Национального банка Республики Беларусь от 8 марта 2026 года [26].

7.5 Вывод об экономической целесообразности реализации проектного решения

Проведенные расчеты технико-экономического обоснования позволяют сделать вывод о перспективности разработки программного средства, предназначенного для реализации инфраструктуры микросервисного приложения агрегатора такси.

Общий объем инвестиций на разработку и реализацию программного средства составляет 20 129,26 белорусских рублей. В качестве модели монетизации выбрана модель подписки, предусматривающая предоставление расширенных возможностей использования программной платформы.

При прогнозируемом объеме продаж 40 платных подписок и отпускной цене 6 000,00 белорусских рублей ожидаемый прирост чистой прибыли организации-разработчика может составить 32 000,00 белорусских рублей в год. Риск недостаточного привлечения пользователей минимизируется цифровым маркетингом и партнерствами.

ЗАКЛЮЧЕНИЕ

Должно быть минимум на 81 странице!!

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга. / К. Ричардсон– СПб. : Питер, 2026. – 544 с.
- [2] Сравнение монолитной и микросервисной архитектуры [Электронный ресурс]. – Режим доступа: <https://proglib.io/p/monolitnaya-vs-mikroservisnaya-arhitektura-2019-09-16> – Дата доступа: 19.02.2026.
- [3] API Gateway Pattern [Электронный ресурс]. – Режим доступа: <https://medium.com/design-microservices-architecture-with-patterns/api-gateway-pattern-8ed0ddfce9df> – Дата доступа: 19.02.2026.
- [4] Service Discovery Pattern [Электронный ресурс]. – Режим доступа: <https://medium.com/design-microservices-architecture-with-patterns/service-registry-pattern-75f9c4e50d09> – Дата доступа: 19.02.2026.
- [5] Circuit Breaker Pattern [Электронный ресурс]. – Режим доступа: <https://www.baeldung.com/cs/microservices-circuit-breaker-pattern> – Дата доступа: 19.02.2026.
- [6] Microservices Asynchronous Message-Based Communication [Электронный ресурс]. – Режим доступа: <https://medium.com/design-microservices-architecture-with-patterns/microservices-asynchronous-message-based-communication-6643bee06123> – Дата доступа: 20.02.2026.
- [7] Apache Kafka [Электронный ресурс]. – Режим доступа: <https://kafka.apache.org> – Дата доступа: 20.02.2026.
- [8] PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs> – Дата доступа: 23.02.2026.
- [9] OAuth 2.0 и OpenID Connect [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/916640/> – Дата доступа: 23.02.2026.
- [10] Identity as a Service [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/companies/mclouds/articles/329964> – Дата доступа: 23.02.2026.
- [11] Spring Authorization Server Reference [Электронный ресурс]. – Режим доступа: <https://docs.spring.io/spring-authorization-server/reference/index.html> – Дата доступа: 23.02.2026.
- [12] Keycloak [Электронный ресурс]. – Режим доступа: <https://www.keycloak.org/guides> – Дата доступа: 23.02.2026.
- [13] Prometheus [Электронный ресурс]. – Режим доступа: <https://prometheus.io/docs/introduction/overview> – Дата доступа: 23.02.2026.
- [14] ELK стек [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/671344> – Дата доступа: 23.02.2026.
- [15] LOKI стек [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/companies/badoo/articles/507718> – Дата доступа: 23.02.2026.
- [16] OpenTelemetry [Электронный ресурс]. – Режим доступа: <https://opentelemetry.io> – Дата доступа: 23.02.2026.
- [17] Podman Documentation [Электронный ресурс]. – Режим доступа: <https://docs.podman.io/en/latest> – Дата доступа: 23.02.2026.
- [18] Docker Documentation [Электронный ресурс]. – Режим доступа:

<https://docs.docker.com> – Дата доступа: 23.02.2026.

[19] Kubernetes Documentation [Электронный ресурс]. – Режим доступа: <https://kubernetes.io/docs/home> – Дата доступа: 23.02.2026.

[20] Docker Compose Documentation [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/compose> – Дата доступа: 23.02.2026.

[21] Docker Swarm Documentation [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/engine/swarm> – Дата доступа: 23.02.2026.

[22] Работа бай [Электронный ресурс]. – Режим доступа: <https://rabota.by> – Дата доступа: 08.03.2026

[23] Официальные курсы белорусского рубля по отношению к иностранным валютам, устанавливаемые Национальным банком Республики Беларусь ежедневно [Электронный ресурс]. – Режим доступа: <https://www.nbrb.by/statistics/rates/ratesdaily> – Дата доступа: 08.03.2026

[24] Производственный календарь на 2026 год [Электронный ресурс]. – Режим доступа: <https://mintrud.gov.by/uploads/files/Proizvodstvennyj-kalendar-2026.pdf> – Дата доступа: 08.03.2026

[25] Об изменении законов по вопросам налоговых правоотношений [Электронный ресурс]. – Режим доступа: <https://pravo.by/document/?guid=12551&p0=N12500127> – Дата доступа: 08.03.2026

[26] Расчетные величины стандартного риска [Электронный ресурс]. – Режим доступа: <https://www.nbrb.by/finsector/financialstability/macprudentialregulation/raschetnye-velichiny-standartnogo-riska> – Дата доступа: 08.03.2026

Вычислительные машины, системы и сети: дипломное проектирование [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_136308.pdf – Дата доступа: 08.03.2026.

Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_161144.pdf – Дата доступа: 08.03.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Код начинается с этого места;

001 Можно с нумерацией строк кода;

002 Шрифт может быть от 12 до 8 pt

ПРИЛОЖЕНИЕ Б
(обязательное)

Спецификация

ПРИЛОЖЕНИЕ В
(обязательное)

Ведомость документов